计算几何专题讲解

EH

2026-10-17

1 引言 4

1.1 精度建模 5

1.2 案例研究 9

1.3 浮点数求和 14

1.4 常用数学函数 16

2 二维计算几何模板 19

2.1 点 20

2.2 直线 37

2.3 线段 51

2.4 多边形 59

2.5 圆 70

3 更多高级算法 77

3.1 凸包切线 78

3.2 闵可夫斯基和 81

3.3 旋转卡壳 83

3.4 半平面交 85

3.5 欧拉公式/皮克定理 90

3.6 随机增量法 91

点击章节或小节标题跳转

下一页目录

返回目录

4 杂题选讲 94

4.1 雷达安装 95
4.2 意大利美食 97
4.3 行对 99
4.4 Dazzling stars 101
4.5 我将如潮水般归来 103
4.7 碰撞伤害 105
4.8 楼观楼观楼楼时间到了 107
4.9 Parallel Lines 110
4.10 博物馆奇妙夜 2 112

5 后记 114

5.1 总结 115
5.2 比赛建议 116
5.3 计算几何综合题单 118
5.4 题单难度与标签 125

References 132

上一页目录

01

引言

返回目录

IEEE-754 提供的精度保障：

能精确表示的整数：

- 对于 `double`，约为 9×10^{15}.
- 对于 `long double`，约为 1.8×10^{19}.
- 特别地，`long double` 类型可以精确执行所有 `long long` 类型可以执行的计算.

浮点计算误差：

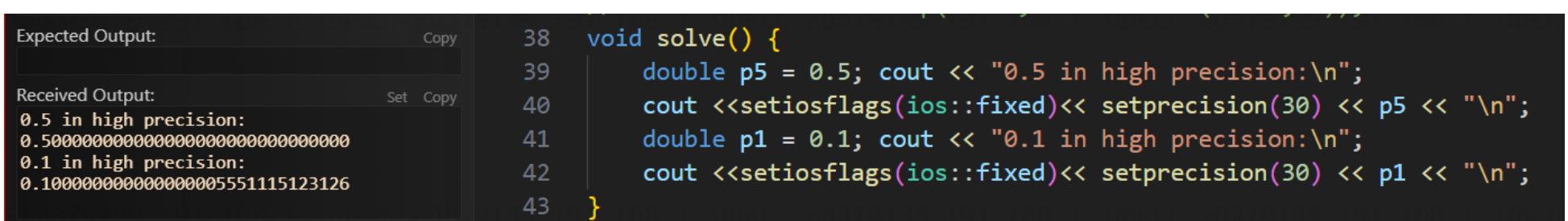
- 每次 $+, -, \times, /, \sqrt{\cdot}$ 等操作都可能引入**相对误差**，记为 ε（machine epsilon）.
- 对于 `double`，$\varepsilon < 1.2 \times 10^{-16}$.
- 对于 `long double`，$\varepsilon < 5.5 \times 10^{-20}$.

再次提醒：这是相对误差.

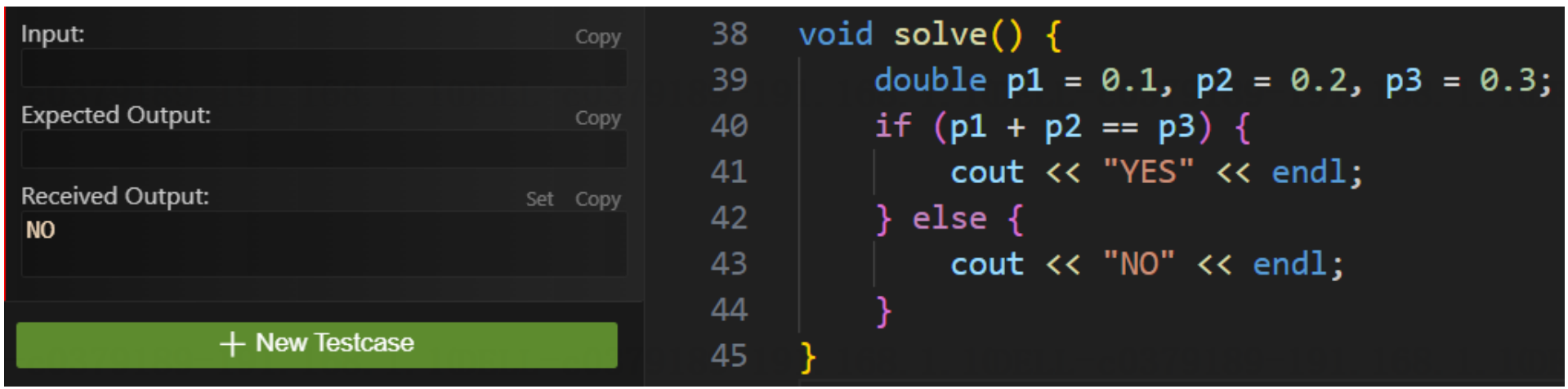
为什么会有误差呢？首先我们要明确一下 double 的存储机制：通过二进制的科学计数法表示。也就是 | sign (1) | exponent (11) | fraction (52) |

形象一点的话，就是 $4 = 100.0_2 = 1.0_2 \times 2^2$

我们清楚，有些数可以使用二进制表示，而有些数不可以。比如 0.5=1/2，0.1 没有。所以在实际情况，是这样的：



对于精度消减情况，也就是误差的产生，源自于 0.1 这种无法被表示完全的数。举个例子：



回顾前页说的，对于整数情况，`long double` 类型可以精确执行所有 `long long` 类型可以执行的计算，这句话应该就不难理解了。

精度爆炸:

一些常见的爆炸情况:

- 一堆数加加减减，先加一个大数最后再减掉，则相对于“最后结果”的相对误差会特别大.
- 加减法累积出的误差，被乘除法加倍放大（e.g. 小心 $1/x$）.
- 一些小误差导致 $\sqrt{\cdot}$、acos(.) 等函数越过定义域而返回 nan.

小 tips:

- 能平方就平方，能不开根就不要开根，开根的误差特别大.
- 使用 acos 的时候记得 clip 一下，-1 到 1.
- 特判一些特殊情况，最好使用整数计算.
- 最好不要用 double 类型的接收整数输入，会很慢.
- 如果是 long double 场景下，sqrt 需要替换为 sqrtl.
- atan2 等函数运行速度很慢，慎用.

Keeping the Dogs Apart（CF101550K）

有两只狗 **A** 和 **B**，分别沿着折线 $A_0, A_1, \ldots, A_n$ 和 $B_0, B_1, \ldots, B_m$ 以相同的速度移动．求两只狗之间的最近距离．

$1 \le n, m \le 10^5$，精度误差 10^{-4}．

还是很好想到需要用双指针扫一下的．

问题简化为做 $n+m$ 次：一只狗在线段 $|PQ|$ 上移动，另一只狗在线段 $|RS|$ 上移动，求最短距离．记为 $\mathrm{minDist}(p, q, r, s)$ 函数（这个函数需要求导去做）

时间复杂度 $O(n+m)$ 或如果使用三分 $O((n+m)\log V)$．

1.2.1 看起来很对但过不去的代码

```
int i = 0, j = 0; // A 和 B 当前所在的线段
double ans = absv(a[0]-b[0]), // 目前的最小距离
       sumA = 0, // A 已完整走过的线段总长
       sumB = 0; // B 已完整走过的线段总长

// 两只狗都还在走
while (i+1 < n && j+1 < m) {
  double start = max(sumA, sumB), // 时间区间起点
         dA = absv(a[i+1]-a[i]), // A 当前线段的长度
         dB = absv(b[j+1]-b[j]), // B 当前线段的长度
         endA = sumA + dA, // A 走完当前线段的时刻
         endB = sumB + dB, // B 走完当前线段的时刻
         end = min(endA, endB); // 时间区间终点

  // 计算两只狗在区间起点和终点的位置
```

```
    pt p = moveBy(a[i], a[i+1], start-sumA),
       q = moveBy(a[i], a[i+1], end-sumA),
       r = moveBy(b[j], b[j+1], start-sumB),
       s = moveBy(b[j], b[j+1], end-sumB);

    // 求这个时间区间内的最近距离
    ans = min(ans, minDist(p,q,r,s));

    // 其中一只狗走到了线段末尾,
    // 于是换到下一段并更新已走过的总长度
    if (endA < endB) {
      i++, sumA += dA;
    } else {
      j++, sumB += dB;
    }
}
```

1.2.2 为什么会这样呢?

对于 `sumA` 和 `sumB` 这两个变量:

- 最多进行 $k = 2 \times 10^5$ 次加法运算.
- 值域最大为 $M = \sqrt{2} \times 10^9$.
- 假设使用 `double`，则 $\varepsilon = 1.2 \times 10^{-16}$.

故绝对误差最大为 $kM\varepsilon \approx 0.032$，超过了 10^{-4} 的精度要求，导致 Wrong Answer.

1.2.3 巧解

此题中，注意到只有 `sumA - sumB` 是有用的．或者说，同时给 `sumA` 和 `sumB` 加上一个常数，不影响计算．

因此，可以在 while 循环体之后加上：

```
double minSum = min(sumA, sumB);
sumA -= minSum;
sumB -= minSum;
```

通过限制值域来减小绝对误差．

当然，你也可以使用 long double 通过。

1.3.1 如何更精确地进行浮点数求和?

刚才的问题中，我们进行了 k 次加法运算，导致相对误差最大为 $k\varepsilon$.

这是因为我们的加法实际上是这样做的:

$$((((a_1 + a_2) + a_3) + a_4) + ...) + a_k$$

能否用类似二叉树的结构，把这些数加起来? 以 $k = 8$ 为例:

$$(((a_1 + a_2) + (a_3 + a_4)) + ((a_5 + a_6) + (a_7 + a_8)))$$

这样“求和次数”就降低为了 $\log k$ 次，相对误差就只有 $\log(k) \cdot \varepsilon$ 了.

刚才那题，使用这一 trick 将绝对误差从 $kM\varepsilon$ 优化成 $\log(k)M\varepsilon$ 也能通过.

当然，这个方法一般万不得已的时候才用，因为一般遇不到这种情况.

1.3.2 封装好的模板

```
struct stableSum {
  int cnt = 0;
  vector<double> v, pref{0};
  void operator+=(double a) {
    assert(a >= 0); // 先保证 a 非负，否则累加不再单调
    int s = ++cnt;
    while (s % 2 == 0) {
      a += v.back();
      v.pop_back(), pref.pop_back();
      s /= 2;
    }
    v.push_back(a), pref.push_back(pref.back() + a);
  }
  double val() { return pref.back(); }
};
```

1.4.1 三角函数与反三角函数

`sin(x)`、`cos(x)`、`tan(x)`：输入弧度制角度，返回对应的三角函数值.

反三角函数 `asin(x)`、`acos(x)`、`atan(x)`：输入函数值，返回弧度制角度.

$$\operatorname{acos}(x) \in [0, \pi], \qquad \operatorname{atan}(x) \in (-\pi/2, \pi/2)$$

注意输入范围：`asin(x)` 和 `acos(x)` 要求 $x \in [-1, 1]$.

$$\operatorname{atan2}(y, x) \in (-\pi, \pi]$$

`atan2(y,x)`：输入点的纵坐标 y 和横坐标 x，返回向量与 x 轴正方向的有向夹角.

相比 `atan(y/x)`，它能正确判断象限，也能处理 $x = 0$ 的情况（原点除外）.

C++ 中使用 `<cmath>`，三角函数默认使用弧度制；角度与弧度转换时要特别注意.

1.4.2 数值处理与精度控制

`sqrt(x)`：输入非负数 x，返回平方根 $\sqrt{x}$，常用于计算向量长度与两点距离.

`fmod(x,y)`：输入浮点数 x, y，返回浮点余数；结果符号与被除数 x 相同，要求 $y \neq 0$.

$$\mathrm{fmod}(7.5, 2.0) = 1.5, \qquad \mathrm{fmod}(-7.5, 2.0) = -1.5$$

`clamp(x,l,r)`：将 x 限制在区间 $[l, r]$ 内，需要包含头文件 `<algorithm>`.

$$\mathrm{clamp}(x, l, r) = \min(\max(x, l), r)$$

高精度输出：使用 `fixed` 和 `setprecision(n)`，需要包含头文件 `<iomanip>`.

```
cout << fixed << setprecision(10) << ans;
```

注意：加上 `fixed` 后，n 表示小数点后位数；否则表示有效数字位数.

1.4.3 python 高精度数值与整数上限

`Fraction`：用分数表示有理数，自动约分，适合需要精确分数运算的场景.

使用 `from fractions import Fraction`，例如 `Fraction(1, 3) + Fraction(1, 6)`，得到精确的 $\frac{1}{2}$.

$$\frac{1}{3}+\frac{1}{6}=\frac{1}{2}$$

`Decimal`：使用 `from decimal import Decimal, getcontext`

通过 `getcontext().prec = 50` 设置精度.

默认精度为 28 位有效数字；设置为 50 后，运算使用 50 位有效数字. 构造时建议传入字符串.

Python 的 `int` 支持任意精度整数，不存在固定的 32 位或 64 位上限，但受内存限制.

02

二维计算几何模板

返回目录

2.1.1 基本定义

```
typedef double T;
struct pt {
  T x, y;
  pt operator+(pt p) {return {x+p.x, y+p.y};}
  pt operator-(pt p) {return {x-p.x, y-p.y};}
  pt operator*(T d) {return {x*d, y*d};}
  pt operator/(T d) {return {x/d, y/d};} // 仅适用于浮点数
  bool operator==(const pt& b){return x == b.x && y == b.y;}
  bool operator!=(const pt& b){return !(*this == b);} // 这两条一般用不到

};
T sq(pt p) {return p.x*p.x + p.y*p.y;} // 求向量长度的平方
T absv(pt p) {return sqrt(sq(p));} // 求向量长度（带误差）
```

提醒：这里的 sqrt，假如是 long double 场景下，需要替换为 sqrtl。

2.1.2 辅助函数

可以使用 `std::ostream` 来定义点的输出方式，方便配合 `std::cerr` 进行调试.

```
ostream& operator<<(ostream& os, pt p) {
  return os << "(" << p.x << "," << p.y << ")";
}
```

用法:

```
pt a = {3,4}, b = {2,-1};
cout << a+b << " " << a-b << "\n"; // (5,3) (1,5)
cout << a*-1 << " " << b/2 << "\n"; // (-3,-4) (1,-0.5)
```

定义 sgn 函数将值域中的数映射到 $\{-1,0,1\}$ 上.

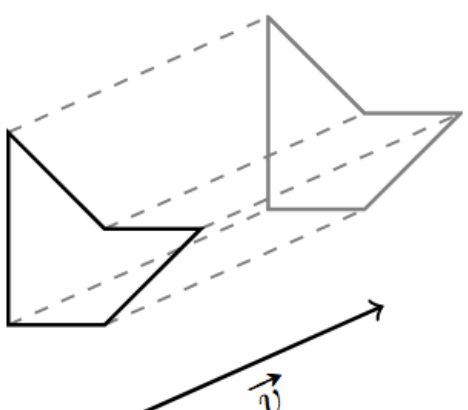
```
int sgn(T x) { return (T(0) < x) - (x < T(0)); }
```

$$\operatorname{sgn}(x)=\begin{cases}-1 & x<0\\ 0 & x=0\\ 1 & x>0\end{cases}$$

2.1.3 基本变换：平移

```
pt translate(pt v, pt p) {
  return p + v;
}
```

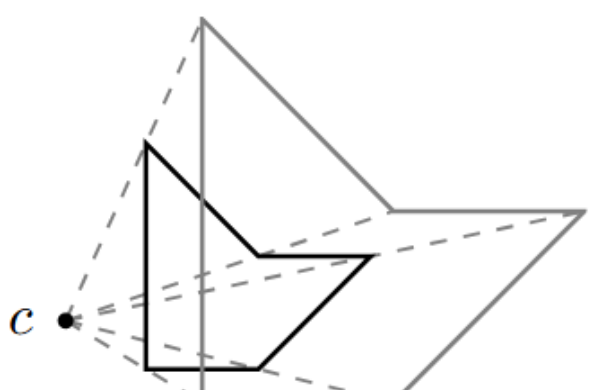
实战一般不用这个函数而是直接加，省事



2.1.4 基本变换：缩放

```
pt scale(pt c, T factor, pt p) {
  return c + (p - c) * factor;
}
```

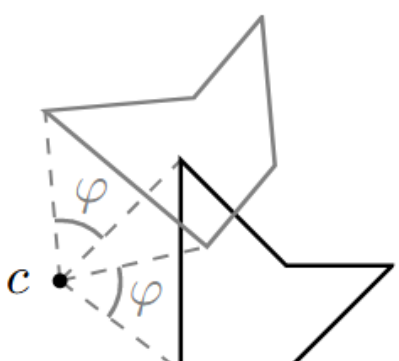
实战一般不用这个函数而是直接加乘，省事



2.1.5 基本变换：旋转（逆时针）

```
pt rot(pt p, T a) {
  return {p.x * cos(a) - p.y * sin(a), p.x * sin(a) + p.y * cos(a)};
}
pt perp(pt p) {
  return {-p.y, p.x};
}
```

我们之后所有的默认排序都是逆时针方向的.

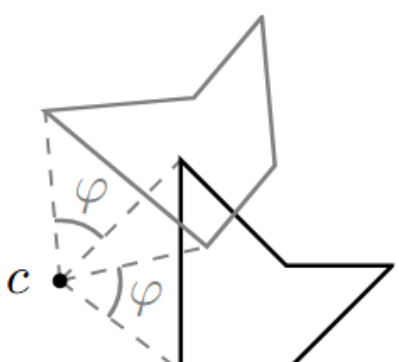


对极坐标熟悉的同学，可以从复数的角度推出这一公式. **（试试看？）**

复数乘法：模长相乘，辐角相加.

逆时针旋转 φ 相当于乘以 $\operatorname{cis}\varphi = \cos\varphi + i\sin\varphi$.

$$\begin{aligned}(x+yi) * \operatorname{cis}\varphi &= (x+yi) * (\cos\varphi + i\sin\varphi) \\ &= (x\cos\varphi - y\sin\varphi) + (x\sin\varphi + y\cos\varphi)i\end{aligned}$$



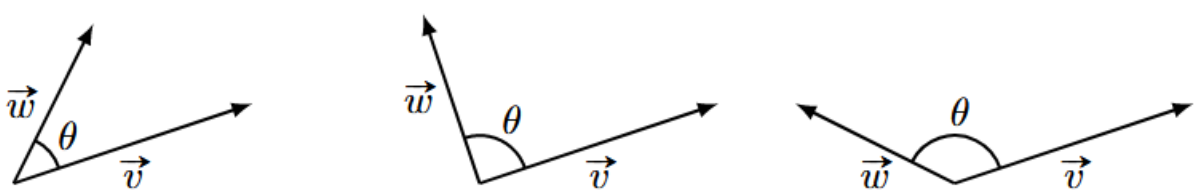
2.1.6 点积（没有顺时针逆时针区别）

点积为一个向量乘以另一个向量在其方向上的投影长度.

因为投影是有方向的，所以我们可以使用点积判断两个向量的夹角是否大于 90 度.

$$\vec{v} \cdot \vec{w} = \|\vec{v}\|\|\vec{w}\| \cos\theta$$

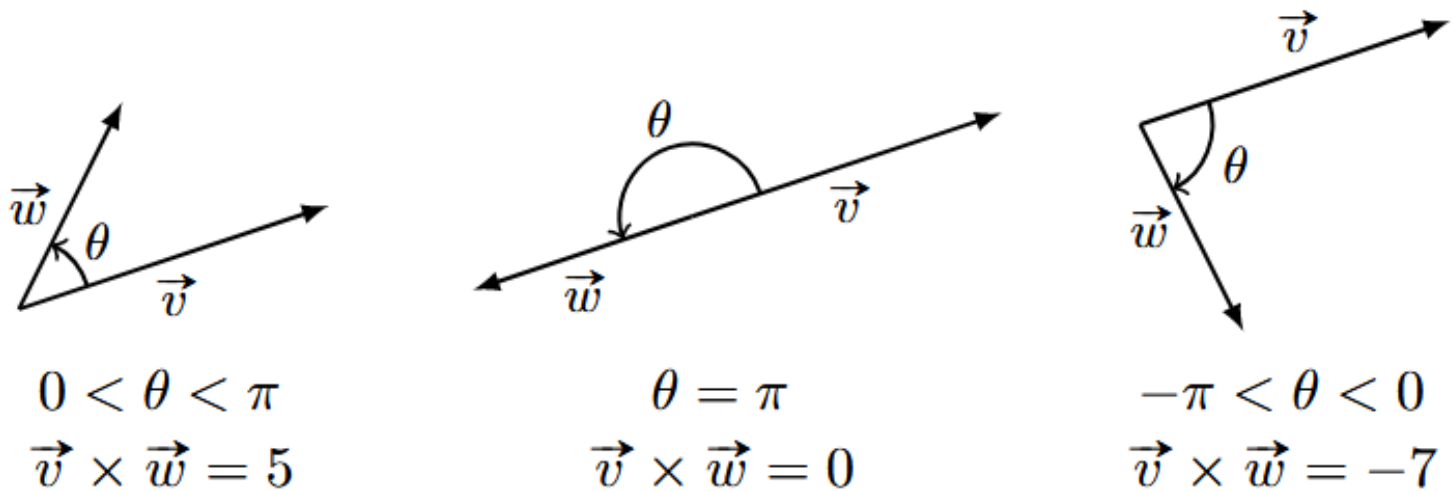
```
T dot(pt v, pt w) { return v.x * w.x + v.y * w.y; }
bool isPerp(pt v, pt w) { return dot(v, w) == 0; }
```



2.1.7 叉积（v 逆时针到 w 角度 <pi 为正）

$$\vec{v} \times \vec{w} = \|\vec{v}\|\|\vec{w}\| \sin\theta$$

```
T cross(pt v, pt w) { return v.x * w.y - v.y * w.x; }
```



$0 < \theta < \pi$
$\vec{v} \times \vec{w} = 5$

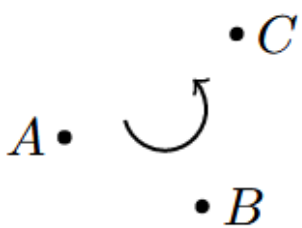
$\theta = \pi$
$\vec{v} \times \vec{w} = 0$

$-\pi < \theta < 0$
$\vec{v} \times \vec{w} = -7$

根据我们高中学的知识（死去的回忆忽然攻击我），这个的 abs 的几何意义是平行四边形面积。所以这个玩意儿除以 2 也就是三角形面积。

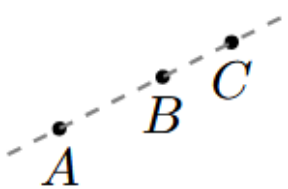
叉积的一个重要作用是判断向量之间的相对关系（一个向量在另一个向量的顺时针一侧还是逆时针一侧）.

```
T orient(pt a, pt b, pt c) { return cross(b - a, c - a); }
```



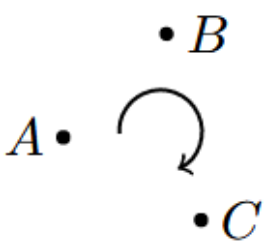
left turn

$\text{orient}(A, B, C) > 0$



collinear

$\text{orient}(A, B, C) = 0$



right turn

$\text{orient}(A, B, C) < 0$

$$\text{orient}(A, B, C) = \text{orient}(B, C, A) = \text{orient}(C, A, B)$$

给定一个多边形零件，把它以 AB 边为底边平放在地上，它左右、上下各占多大空间?

这其实就是一个外接矩形问题：所求空间是矩形的面积 长 $\times$ 宽.

求长（水平跨度）：

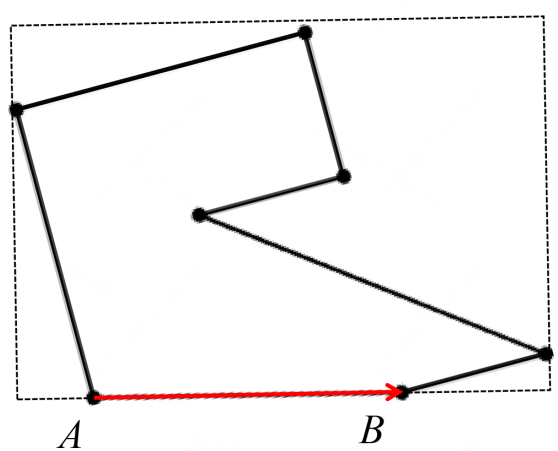
把底边方向记为向量 $\vec{u} = B - A$，任取一点 P_i，用点积求它在 $\vec{u}$ 上的投影 $t_i = \vec{u} \cdot P_i$. 遍历所有顶点，取最大值与最小值之差：

$$\text{长} = \frac{\max_i t_i - \min_i t_i}{\|\vec{u}\|}$$

求宽（竖直跨度）：

对每个顶点 P_i，计算叉积 $s_i = \vec{u} \times (P_i - A)$，

则 宽 $= \dfrac{\max_i s_i - \min_i s_i}{\|\vec{u}\|}$，两者相乘即为答案.



可以用点积求向量夹角.

```
T angle(pt v, pt w) {
  T cosTheta = dot(v,w) / absv(v) / absv(w);
  return acos(max(-1.0, min(1.0, cosTheta)));
}
```

注意 acos 函数对定义域 $[-1,1]$ 之外的输入会返回 nan，请手动 clip 到 $[-1,1]$ 的范围内.

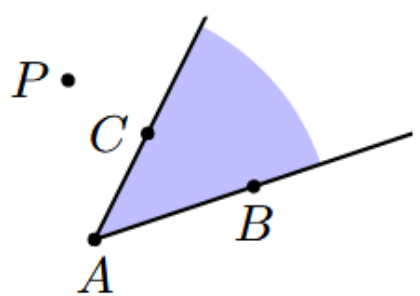
或者使用 atan2，可以规避 acos.

```
T angle(pt v, pt w) {
  return abs(atan2(cross(v, w), dot(v, w)));
}
```

这个函数只返回 0-pi 的角度，纯粹的夹角，带方向的在下面的 `orientedAngle`

再利用 $\text{orient}(A, B, C)$ 函数我们可以判断另一个点是否在两个向量所夹的区域内.

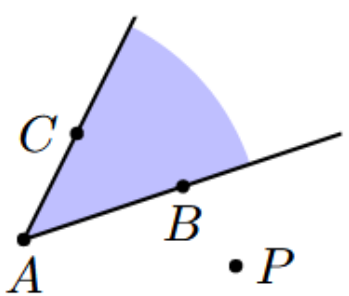
```
bool inAngle(pt a, pt b, pt c, pt p) {
  assert(orient(a, b, c) != 0);
  if (orient(a, b, c) < 0) swap(b, c);
  return orient(a, b, p) >= 0 && orient(a, c, p) <= 0;
}
```



$\text{orient}(A, B, P) \geq 0$

$\text{orient}(A, C, P) > 0$

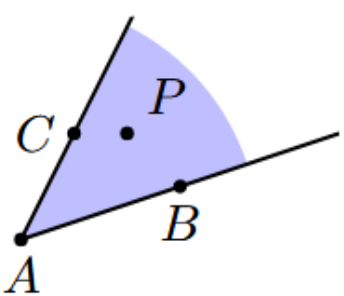
KO



$\text{orient}(A, B, P) < 0$

$\text{orient}(A, C, P) \leq 0$

KO



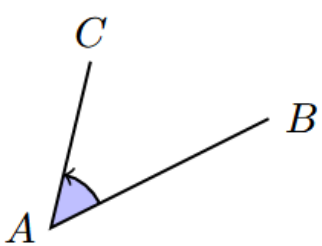
$\text{orient}(A, B, P) \geq 0$

$\text{orient}(A, C, P) \leq 0$

OK

注意我们使用 angle 函数求解的角度，可以利用叉积正负性将其升级为带方向的 orientedAngle 函数. 然后我们可以尝试写一个 inorientedAngle 函数.（**试试看？**）

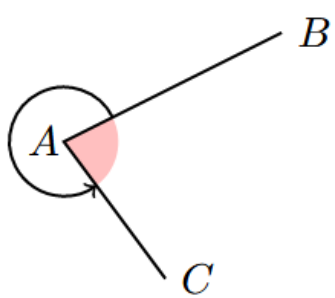
```
T orientedAngle(pt a, pt b, pt c) {
  if (orient(a, b, c) >= 0) { return angle(b - a, c - a); }
  else { return 2 * M_PI - angle(b - a, c - a);
}
```



$\text{orient}(A,B,C) > 0$

angle() $= 50°$

orientedAngle() $= 50°$



$\text{orient}(A,B,C) < 0$

angle() $= 80°$

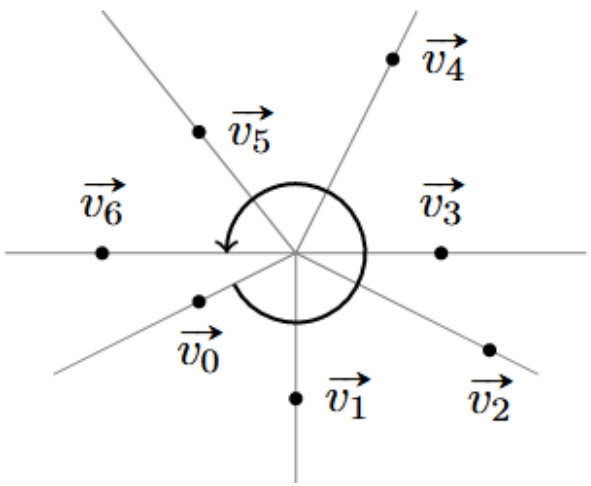
orientedAngle() $= 280°$

```
bool inOrientedAngle(pt a, pt b, pt c, pt p) {
    T x = orient(a, b, p);
    T y = orient(a, c, p);
    T z = orient(a, b, c);

    if (z >= 0) return x >= 0 && y <= 0;
    else return x >= 0 || y <= 0;
}
```

2.1.8 极角排序

极角排序是一个经典问题，考察我们是否能以原点为基准，将所有点按照逆时针顺序进行排序.

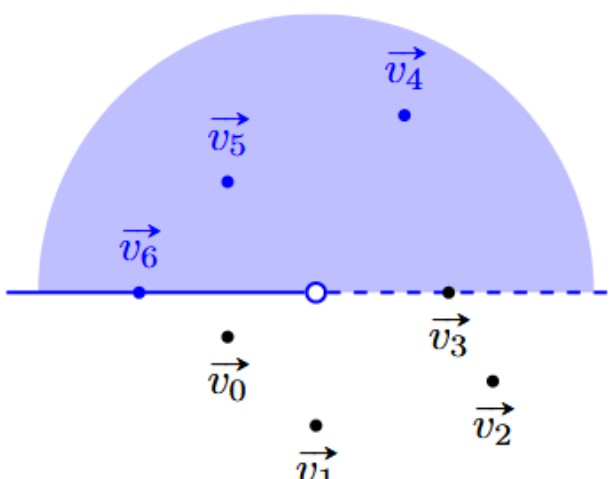


最经典的做法是使用 `atan2` 函数，但其精度可能不够安全……

希望通过叉积将这些向量进行排序.

在 $\vec{v}$ 和 $\vec{w}$ 是相近的向量时（准确来说，他们在逆时针顺序上的有向角度 $< 180°$），则 $\vec{v}$ 在 $\vec{w}$ 前面的充要条件是 $\vec{v} \times \vec{w} > 0$.

如果把所有向量分成到两个半平面上分别做，则两个半平面内部待排序的角度都不会超过 $180°$，就可以正常判断了！



这是一个多关键字排序.

```
bool half(pt p) { // 是否在蓝色半平面内
  assert(p.x != 0 || p.y != 0); // (0,0) 的极角没有定义
  return p.y > 0 || (p.y == 0 && p.x < 0);
}
void polarSort(vector<pt> &v) {
  sort(v.begin(), v.end(), [](pt v, pt w) {
    return make_tuple(half(v), 0, sq(v)) <
           make_tuple(half(w), cross(v,w), sq(w));
  });
}
```

如果两点与原点共线，则距原点较近的点会被排在前面.

注意：此处使用的 comparator 不是良序的，需要保证 cross 的正、负、零判定可靠.

实战中多为整数坐标，使用 long double 或者 long long 即可.

2.2.1 基本定义

用方向向量 v 和偏移量 c 来表示直线.

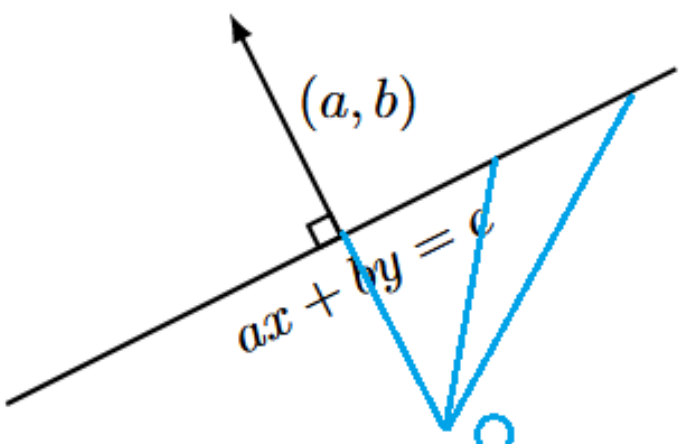
```
struct line {
  pt v; T c;
  // 由方向向量 v 和偏移量 c 构造
  line(pt v, T c) : v(v), c(c) {}
  // 由方程 ax+by=c 构造
  line(T a, T b, T c) : v({b,-a}), c(c) {}
  // 由两点 P 和 Q 构造
  line(pt p, pt q) : v(q-p), c(cross(v,p)) {}
}
```

这是怎么来的?

数学上，我们通常使用 $ax + by = c$ 来表示直线.

取垂直于直线的方向向量 (a, b)，则直线方程可写成 $(a, b) \cdot (x, y) = c$. 表示垂直于方向向量 (a, b) 的一系列点.

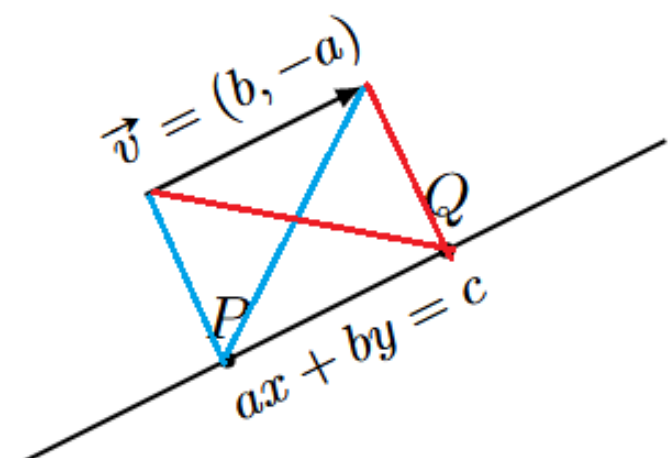
从原点到这些点的连线到 (a, b) 这一方向向量的投影长度均相同，为 $\dfrac{c}{\sqrt{a^2 + b^2}}$.



也可以从叉积的角度理解，取方向向量 $(b, -a)$，则直线方程可写成 $(b, -a) \times (x, y) = c$.

其中这个方向向量就是直线的方向向量.

从几何的角度理解，选取 $(0, 0)$，$(b, -a)$ 和直线上任一点所构成的三角形面积均相同.



如果给定直线上两个点 (x_1, y_1) 和 (x_2, y_2)，想要初始化直线，则：

$$(b, -a) = (x_2, y_2) - (x_1, y_1)$$

然后因为直线上任一点都满足那个叉积形式的等式：

$$(b, -a) \times (x, y) = (b, -a) \times (x_1, y_1) = c$$

可得：

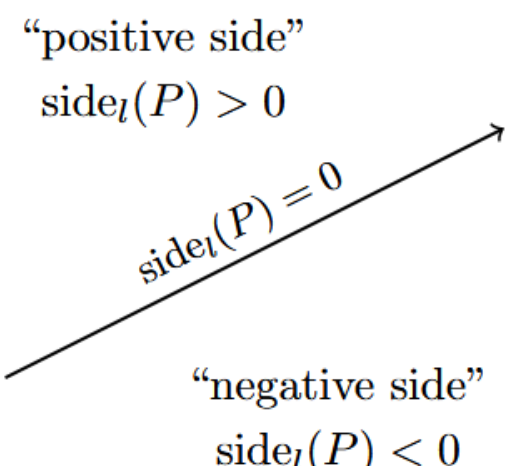
$$c = (x_2 - x_1, y_2 - y_1) \times (x_1, y_1)$$

以上，就是前面给的代码中的三种初始化方式.

2.2.2 点与直线的位置关系

我们的直线给定了方向向量，据此，我们将直线所划分为的两个半平面分别定义为“postivie side”和“negative side”.

$$\text{side}_l(P) := ax + by - c = \vec{v} \times P - c$$

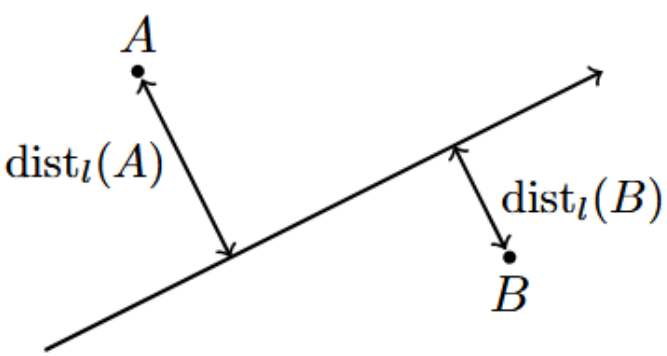



```
T side(line ln, pt p) { return cross(ln.v, p) - ln.c; }
```

2.2.3 点到直线的距离

如果你熟悉叉积的性质的话，又不难发现：我们可以通过 `side` 函数简单的得到 `dist` 函数. ——这就是叉积的性质.

$$\mathrm{dist}_l(P) = \frac{\mathrm{side}_l(P)}{\|\vec{v}\|}$$




```
T dist(line ln, pt p) { return abs(side(ln, p)) / absv(ln.v); }
```

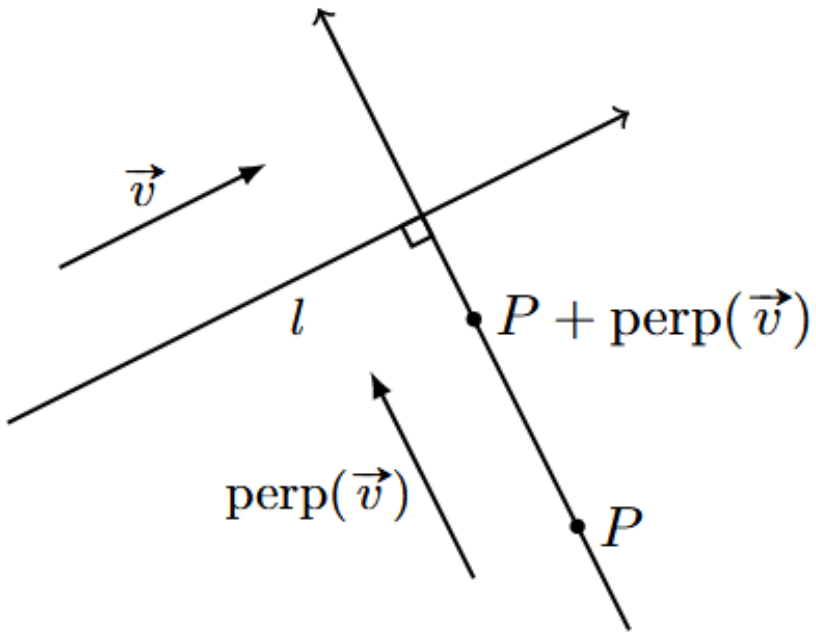
一个值得注意的细节是，在检查点是否在直线的某个整数距离内时，建议使用 `sqDist` 方法.

```
T sqDist(line ln, pt p) { return side(ln, p) * side(ln, p) / sq(ln.v); }
```

虽然用到了 `double` ……

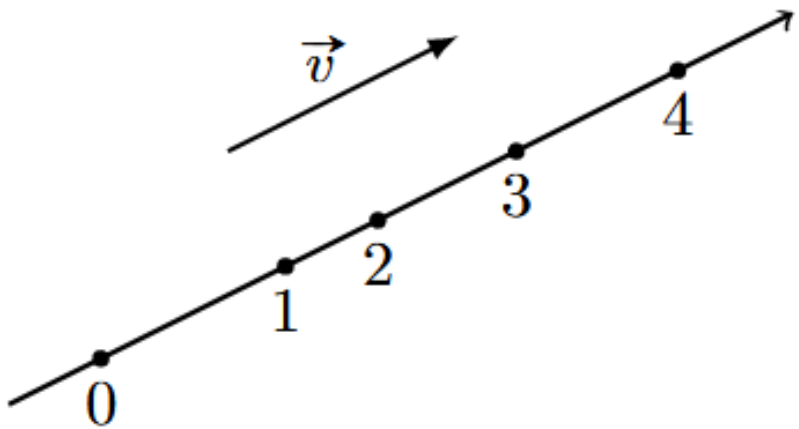
回顾我们学到的 IEEE-754 浮点数性质：如果操作数和运算结果都在给定范围内且都是整数，则运算被保证是精确的.

2.2.4 过一点的垂线




```
line perpThrough(line ln, pt p) { return {p, p + perp(ln.v)}; }
```

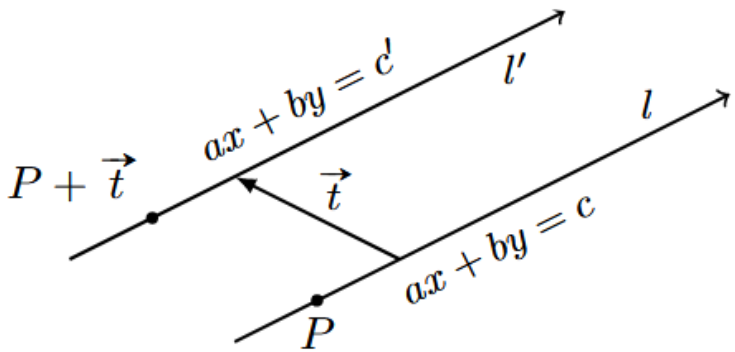
2.2.5 沿直线排序




```
bool cmpProj(line ln, pt p, pt q) { return dot(ln.v, p) < dot(ln.v, q); }
```

基于这个比较函数调用 `std::sort` 即可.

2.2.6 沿指定方向平移



可以用 $\text{side}_l(P)$ 来判断点是否在直线上！

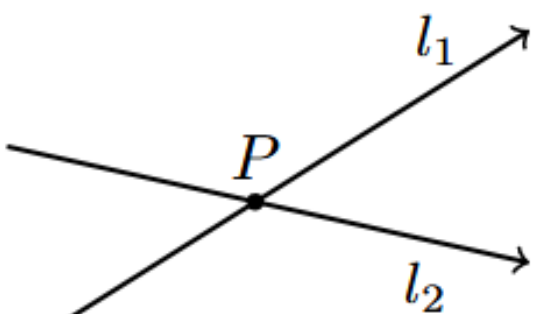
$$\begin{cases} \text{side}_l(P) = \vec{v} \times P - c = 0 \\ \text{side}_{l'}(P + \vec{t}) = \vec{v} \times (P + \vec{t}) - c' = 0 \end{cases} \Longrightarrow c' = \vec{v} \times (P + \vec{t}) = c + \vec{v} \times \vec{t}$$

```
line translate(line ln, pt t) { return {ln.v, ln.c + cross(ln.v, t)}; }
```

2.2.7 直线交点

任意两条不平行（$\vec{v}_1 \times \vec{v}_2 \neq 0$）的直线都存在唯一交点：

$$P = \frac{c_1\vec{v}_2 - c_2\vec{v}_1}{\vec{v}_1 \times \vec{v}_2}$$

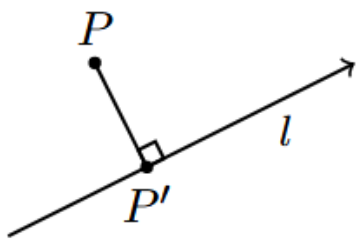


可以用 side 证明：

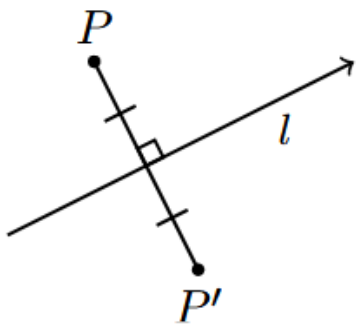
$$\begin{aligned}\text{side}_{l_1}(P) &= \vec{v}_1 \times \left(\frac{c_1\vec{v}_2 - c_2\vec{v}_1}{\vec{v}_1 \times \vec{v}_2}\right) - c_1 \\ &= \frac{c_1(\vec{v}_1 \times \vec{v}_2) - c_2(\vec{v}_1 \times \vec{v}_1) - c_1(\vec{v}_1 \times \vec{v}_2)}{\vec{v}_1 \times \vec{v}_2} \\ &= \frac{-c_2(\vec{v}_1 \times \vec{v}_1)}{\vec{v}_1 \times \vec{v}_2} = 0\end{aligned}$$

```
bool inter(line l1, line l2, pt &out) {
  T d = cross(l1.v, l2.v);
  if (d == 0) return false;
  out = (l2.v * l1.c - l1.v * l2.c) / d;
  return true;
}
```

2.2.8 正交投影与反射



projection



reflection

相当于求解因子 k，使得：

$$\text{side}_l(P + k\,\text{perp}(\vec{v})) = 0$$

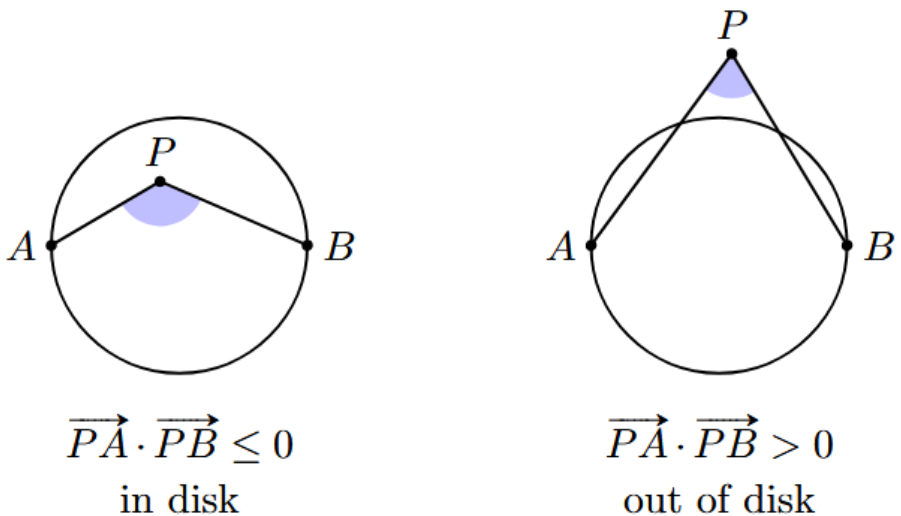
$$\begin{aligned}\text{side}_l(P + k\,\text{perp}(\vec{v})) &= \vec{v} \times (P + k\,\text{perp}(\vec{v})) - c \\ &= \vec{v} \times P + \vec{v} \times k\,\text{perp}(\vec{v}) - c \\ &= (\vec{v} \times P - c) + k(\vec{v} \times \text{perp}(\vec{v})) \\ &= \text{side}_l(P) + k\|\vec{v}\|^2\end{aligned}$$

$$\implies \quad k = -\frac{\text{side}_l(P)}{\|\vec{v}\|^2}$$

反射的话就是因子换成 $2k$.

```
pt proj(line ln, pt p) { return p - perp(ln.v) * side(ln, p) / sq(ln.v); }
pt refl(line ln, pt p) { return p - perp(ln.v) * 2 * side(ln, p) / sq(ln.v); }
```

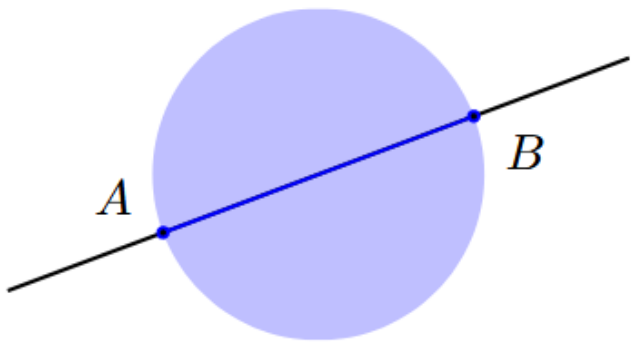
2.3.1 点在线段为直径的圆上




```
bool inDisk(pt a, pt b, pt p) { return dot(a - p, b - p) <= 0; }
```

2.3.2 点在线段上

如果只判断叉积为 0 是不对的，可以用 `inDisk` 函数辅助判断.



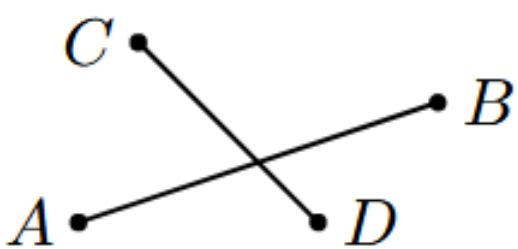
intersection of line and disk = segment

```
bool onSegment(pt a, pt b, pt p) {
  return orient(a, b, p) == 0 && inDisk(a, b, p);
}
```

2.3.3 线段与线段交点

线段与线段之间的交点有两种：

- 一种是正常的：类似于直线相交的情况.
- 另一种是退化的：可能在端点处相交，或者整个线段重合.



proper intersection

先判断有没有，有的话再通过加权平均（相当于在 A 点的权重为 $1/o_a$，B 点的权重为 $1/o_b$）得到.

```
bool properInter(pt a, pt b, pt c, pt d, pt &out) {
  T oa = orient(c, d, a), ob = orient(c, d, b),
    oc = orient(a, b, c), od = orient(a, b, d);
  // 规范相交存在的充要条件是两侧符号相反
  if (oa * ob < 0 && oc * od < 0) {
    out = (a * ob - b * oa) / (ob - oa);
    return true;
  }
  return false;
}
```

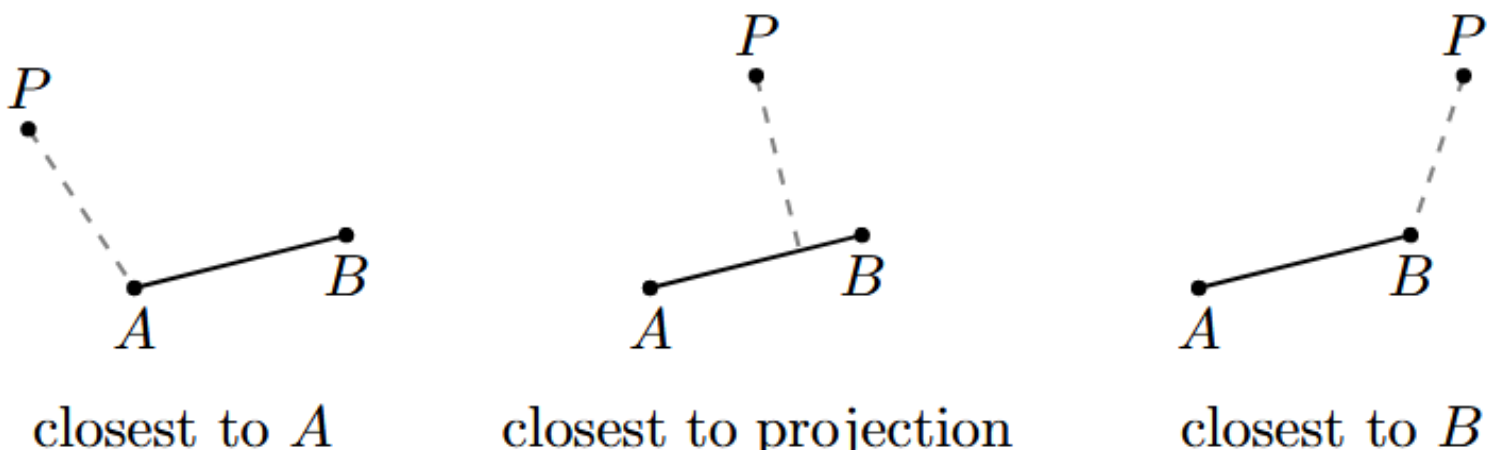
再处理退化的情况.

```
set<pt> inters(pt a, pt b, pt c, pt d) {
  pt out;
  if (properInter(a, b, c, d, out)) return {out};
  set<pt> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return s;
}
```

如果 $|s| = 1$ 则只有一个交点；如果 $|s| = 2$ 则说明相交的区域是一条线段，并且 s 中存的是其两个端点.

2.3.4 点到线段的距离

得看垂足有没有落在线段上，否则就是到端点的距离了.

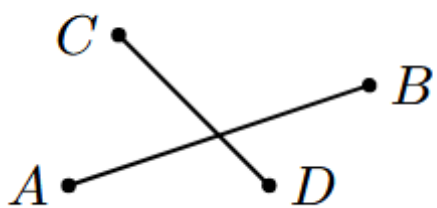


记得处理线段退化成一个点的情况.

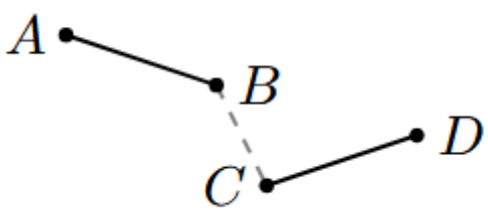
```
T segPoint(pt a, pt b, pt p) {
  if (a != b) {
    line l(a, b);
    if (cmpProj(l, a, p) && cmpProj(l, p, b)) // 若垂足落在线段上
      return dist(l, p);                      // 返回到直线的距离
    return min(absv(p - a), absv(p - b));   // 否则返回到 A 或 B 的距离
  }
  return absv(p - a);
}
```

2.3.5 线段到线段的距离

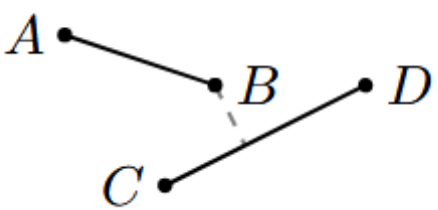
要么直接相交，距离为 0；否则一定可以退化到点到线段距离的 case.



proper intersection



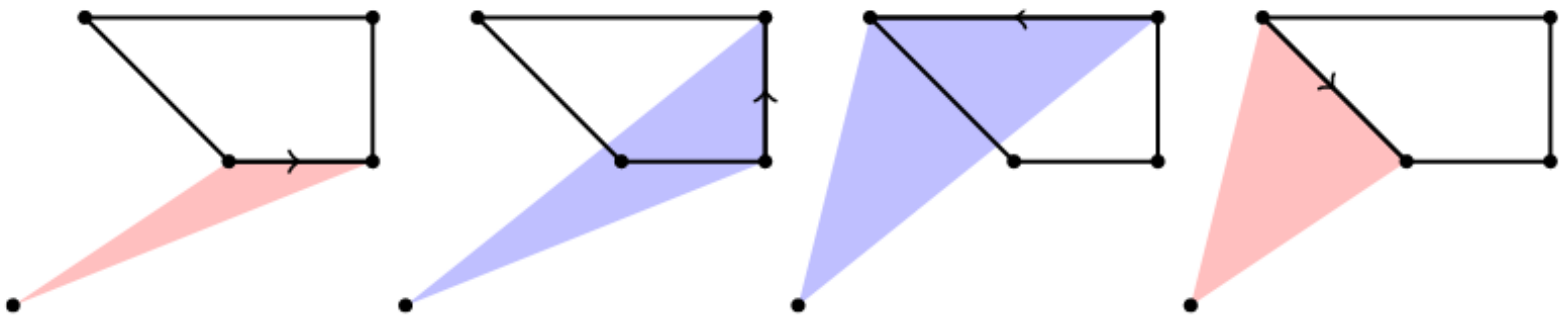
between points



between point and line

```
T segSeg(pt a, pt b, pt c, pt d) {
  pt dummy;
  if (properInter(a, b, c, d, dummy)) return 0;
  return min({segPoint(a, b, c), segPoint(a, b, d),
              segPoint(c, d, a), segPoint(c, d, b)});
}
```

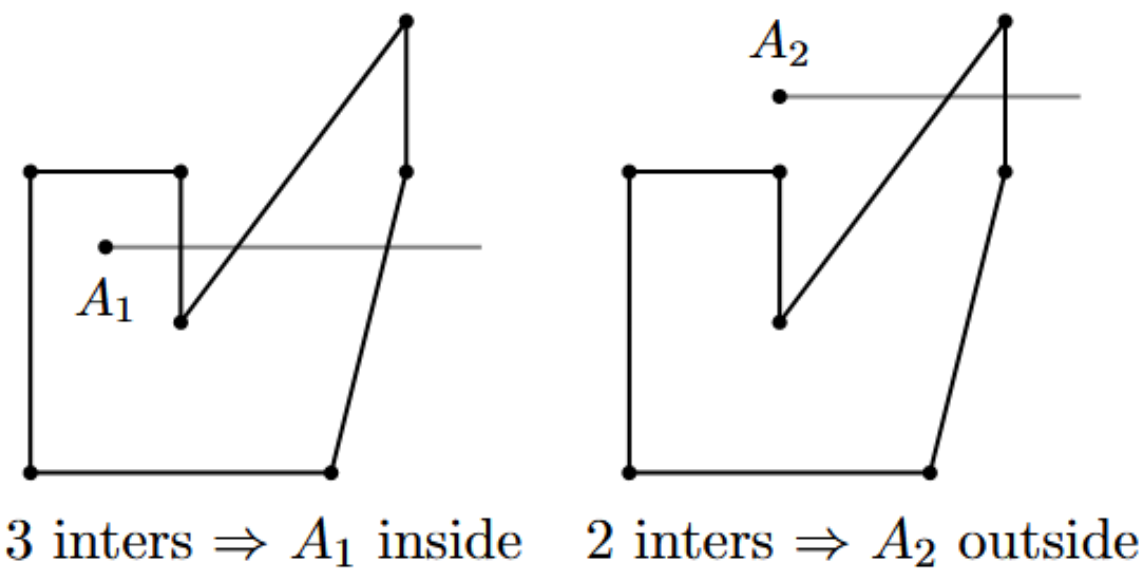
2.4.1 多边形面积



```
T areaPolygon(vector<pt> p) {
  T area = 0.0;
  for (int i = 0, n = p.size(); i < n; i++) {
    area += cross(p[i], p[(i + 1) % n]); // i == n-1 时回绕到 0
  }
  return abs(area) / 2.0;
}
```

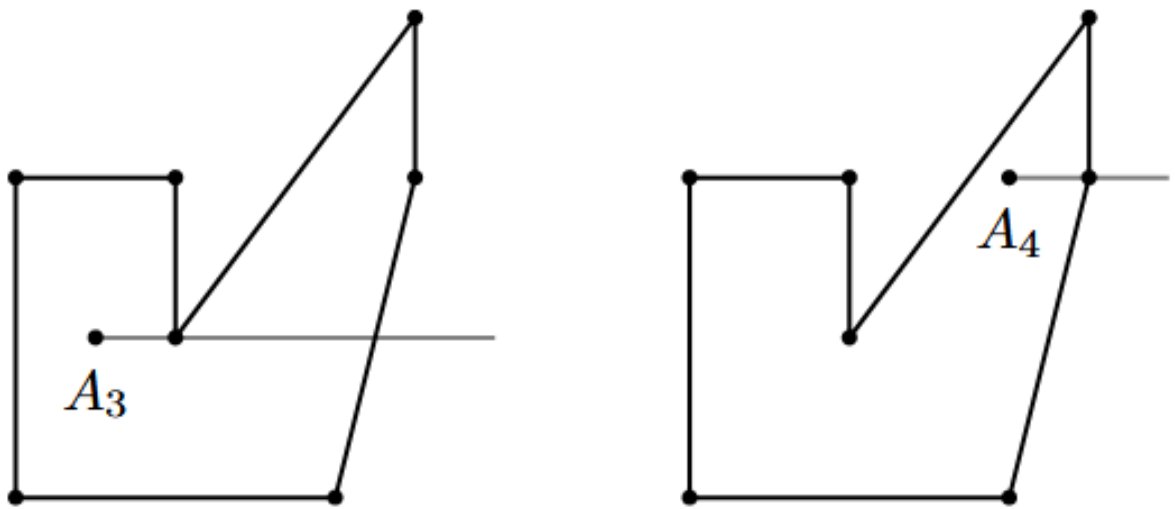
2.4.2 Cutting-ray Test

判断点是否在多边形内部的方法：从该点出发引一条直线，数该直线与多边形相交的次数.



这个“相交”有很多细节：

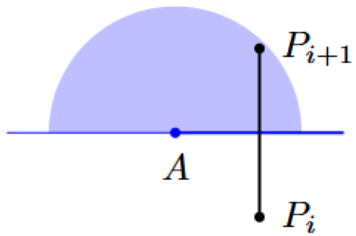
端点到底是算还是不算……



1. 线段必须穿过射线.
2. 线段的两个端点必须一个位于半平面内，一个位于半平面外（这里的半平面包括 x 轴上的点）.

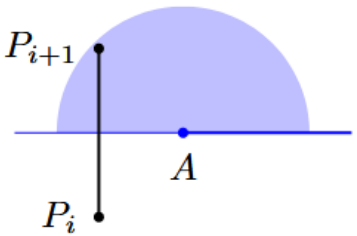
```
// P 是否至少与 A 一样高（蓝色部分）
bool above(pt a, pt p) {
  return p.y >= a.y;
}

// 判断射线是否穿过线段 PQ
bool crossesRay(pt a, pt p, pt q) {
  return (above(a, q) - above(a, p)) *
           orient(a, p, q) > 0;
}
```



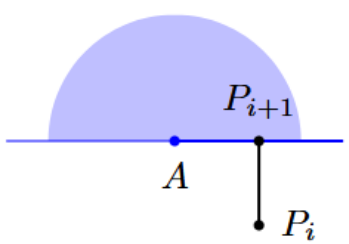
- touches ray: OK
- halves $\neq$: OK

$\Rightarrow$ crossing



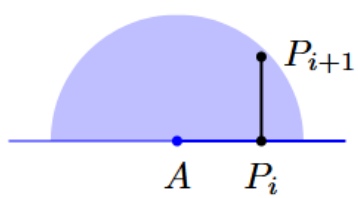
- touches ray: KO
- halves $\neq$: OK

$\Rightarrow$ no crossing



- touches ray: OK
- halves $\neq$: OK

$\Rightarrow$ crossing



- touches ray: OK
- halves $\neq$: KO

$\Rightarrow$ no crossing

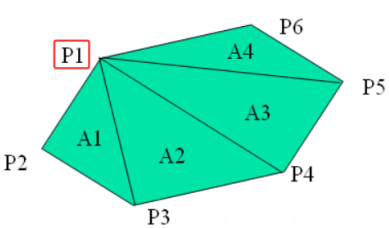
通过对 crossesRay 计数来判断点是否在多边形内. 另外提供了一个额外选线以允许/拒绝点恰好在多边形的边上.

```
// strict 为 true 时, A 落在边界上返回 false
bool inPolygon(vector<pt> p, pt a, bool strict = true) {
  int numCrossings = 0;
  for (int i = 0, n = p.size(); i < n; i++) {
    if (onSegment(p[i], p[(i + 1) % n], a)) return !strict;
    numCrossings += crossesRay(a, p[i], p[(i + 1) % n]);
  }
  return numCrossings & 1; // 交点个数为奇数则在多边形内
}
```

另一种方法是使用 Winding Number, 可自行学习.

对于已经逆时针排好的凸包的点（CCW），有 log 级别的判别方式（严格）：

```
bool pointInConvexStrict(const vector<pt>&A, const pt&p) {
  int n = (int)A.size();
  if (n<3) return false;
  if (sgn(orient(A[0], A[1], p)) <= 0) return false;
  if (sgn(orient(A[0], A[n-1], p)) >= 0) return false;
  int l = 1, r = n-1;
  while (r-l > 1) {
    int mid = (l+r) >>1;
    if (sgn(orient(A[0], A[mid], p)) > 0) l = mid;
    else r=mid;
  }
  return sgn(orient(A[l], A[l+1], p)) > 0;}
```

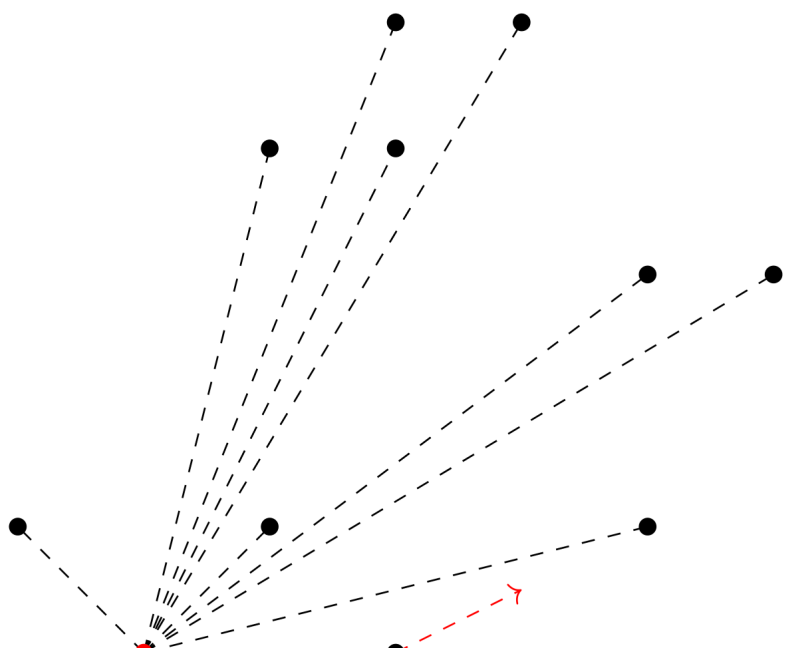


假如想让他允许在边上，把标黄的部分添加/减少等于号就行

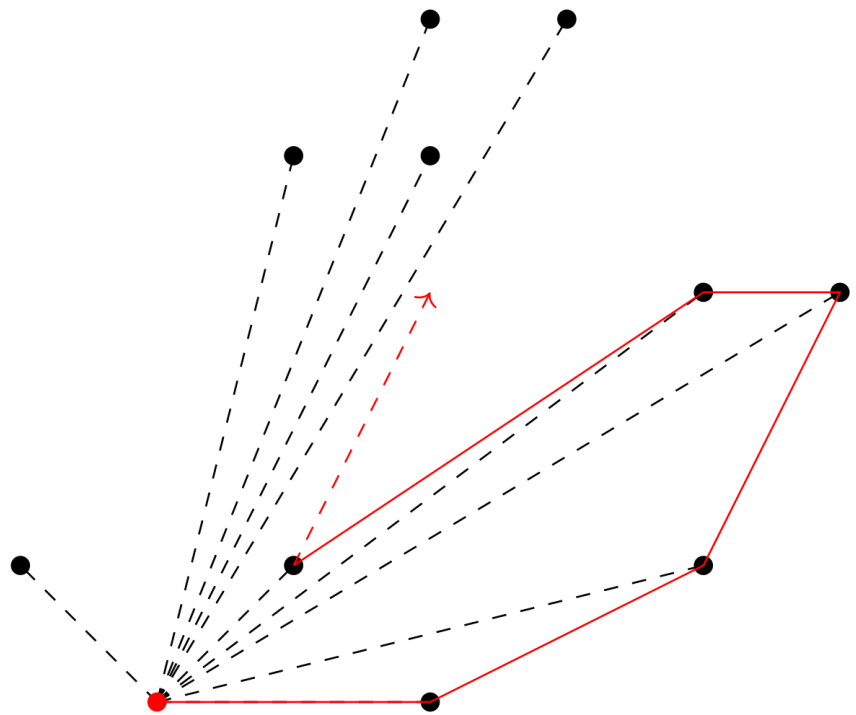
2.4.3 凸包

如何求一堆点集的凸包？极角排序后用一个栈维护

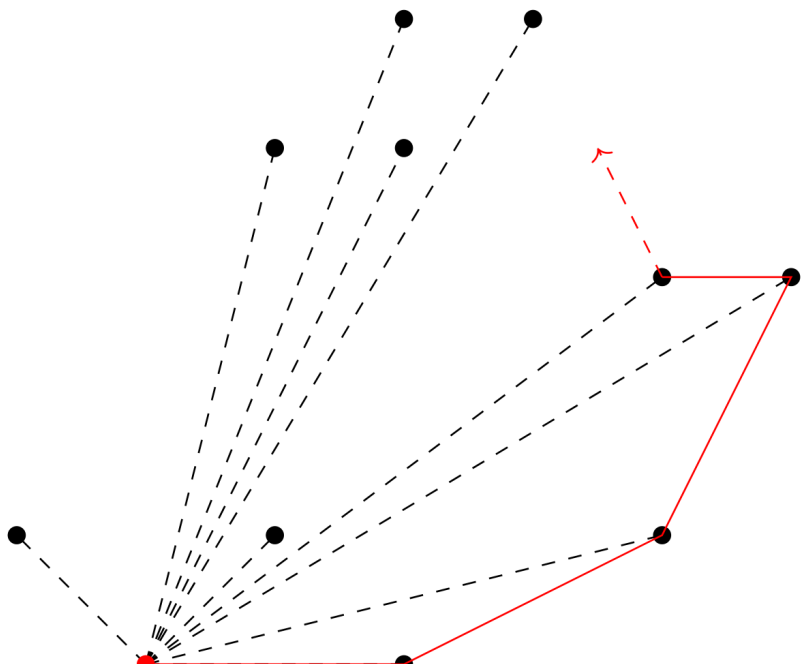
每次寻找栈头直线左边的点入栈，遇到右边的点就弹栈，直到栈空或者栈顶在直线左边.



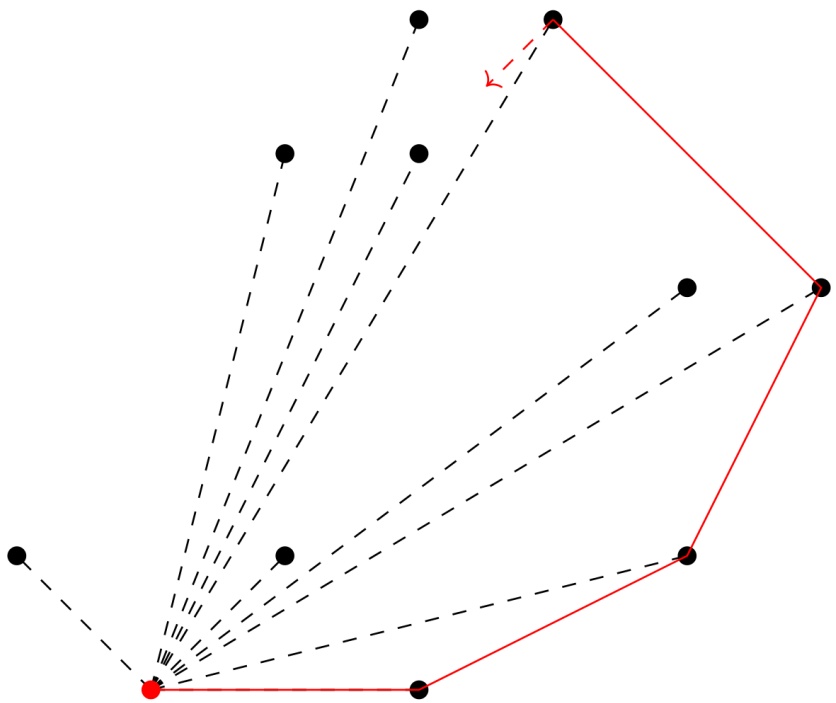
按理来说这个凸包应该一直在“turn left”，如果没有的话就从栈里 pop 掉节点.



可能弹了一个继续不行……

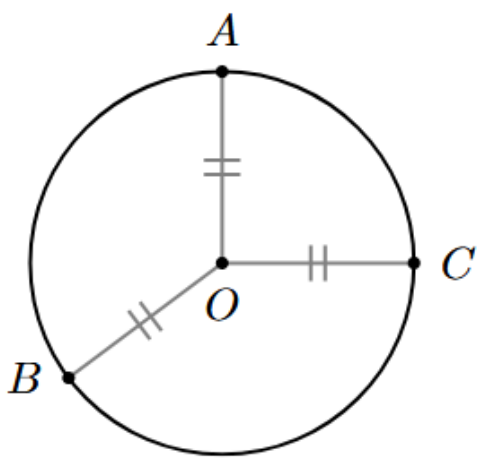


这下终于对啦!



```
bool turn_left(pt a, pt b, pt c) { return sgn(cross(b - a, c - a)) > 0; }
vector<pt> convex_hull(vector<pt> a) {
  if (a.size() <= 2) return a;
  pt base = *min_element(a.begin(), a.end());
  sort(a.begin(), a.end(), [&](auto u, auto v) {
    return make_tuple(half(u - base), 0, sq(u - base)) <
           make_tuple(half(v - base), cross(u - base, v - base), sq(v - base));
  });
  vector<pt> ret;
  for (auto i : a) {
    while (ret.size() > 1 && !turn_left(ret[ret.size()-2],ret[ret.size()-1],i))
       ret.pop_back();
    ret.push_back(i);
  }
  return ret;
}
```

2.5.1 外接圆



外接圆圆心是线段 AB 和 AC 的中垂线的交点.

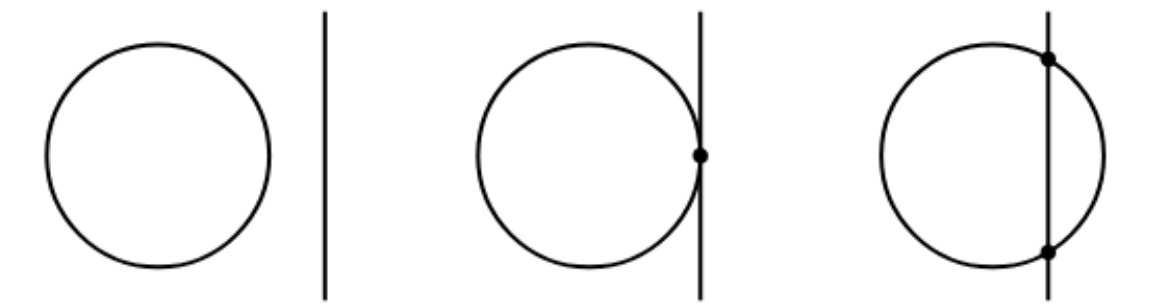
线段 $|AB|$ 的方向向量是 $\mathrm{perp}(\overrightarrow{AB})$，中点是 $\frac{1}{2}\overrightarrow{AB}$.

$$\begin{aligned}\overrightarrow{AO} &= \frac{\left(-\frac{1}{2}|AB|^2\right)\operatorname{perp}(\overrightarrow{AC}) - \left(-\frac{1}{2}|AC|^2\right)\operatorname{perp}(\overrightarrow{AB})}{\operatorname{perp}(\overrightarrow{AB}) \times \operatorname{perp}(\overrightarrow{AC})} \\ &= \frac{\operatorname{perp}\left(|AC|^2\overrightarrow{AB} - |AB|^2\overrightarrow{AC}\right)}{2\overrightarrow{AB} \times \overrightarrow{AC}}\end{aligned}$$

```
pt circumCenter(pt a, pt b, pt c) {
  b = b - a, c = c - a; // 换成以 A 为原点的坐标
  assert(cross(b, c) != 0); // A、B、C 共线时没有外接圆
  return a + perp(b * sq(c) - c * sq(b)) / cross(b, c) / 2;
}
```

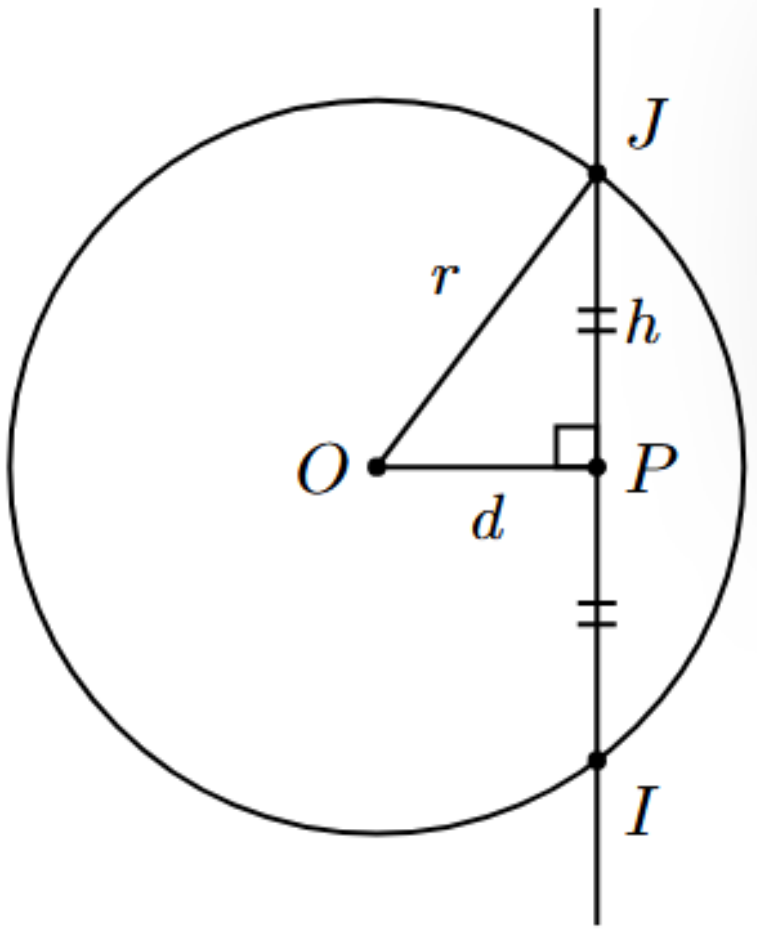
2.5.2 圆与直线的交点

根据圆心到直线的距离就容易判断交点到底有几个.

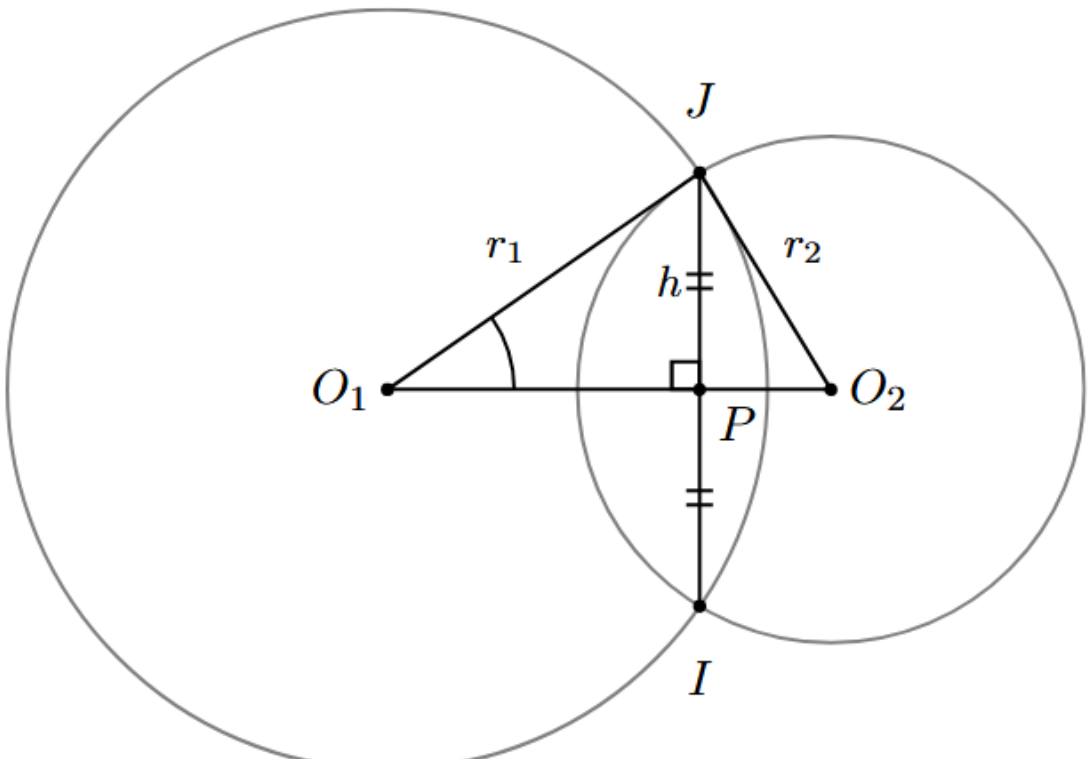


$$h = \sqrt{r^2 - d^2}$$

```
int circleLine(pt o, T r, line ln,pair
<pt, pt> &out) {
  T h2 = r * r - sqDist(ln, o);
  if (h2 >= 0) {
    pt p = proj(ln, o);
    pt h = ln.v * sqrt(h2) / absv(ln.v);
    out = {p - h, p + h};
  }
  return 1 + sgn(h2);
}
```



2.5.3 圆与圆的交点



设 $d = |O_1O_2|$. 对三角形 O_1O_2J 使用余弦定理，可得

$$\cos(\angle O_2O_1J) = \frac{d^2 + r_1^2 - r_2^2}{2dr_1}$$

又因为 O_1PJ 是直角三角形，所以

$$|O_1P| = r_1 \cos(\angle O_2O_1J) = \frac{d^2 + r_1^2 - r_2^2}{2d}$$

由此即可求出点 P 的位置.

接下来求 $h = |PI| = |PJ|$. 对三角形 O_1PJ 使用勾股定理，

得到 $h = \sqrt{r_1^2 - |O_1P|^2}$.

```
int circleCircle(pt o1, T r1, pt o2, T r2, pair<pt, pt> &out) {
  pt d = o2 - o1;
  T d2 = sq(d);
  if (d2 == 0) {
    assert(r1 != r2);
    return 0; // 同心圆
  }
  T pd = (d2 + r1 * r1 - r2 * r2) / 2; // = |O_1P| * d
  T h2 = r1 * r1 - pd * pd / d2;               // = h^2
  if (h2 >= 0) { pt p = o1 + d * pd / d2, h = perp(d) * sqrt(h2 / d2); out = {p
- h, p + h}; }
  return 1 + sgn(h2);
}
```

03

更多高级算法

返回目录

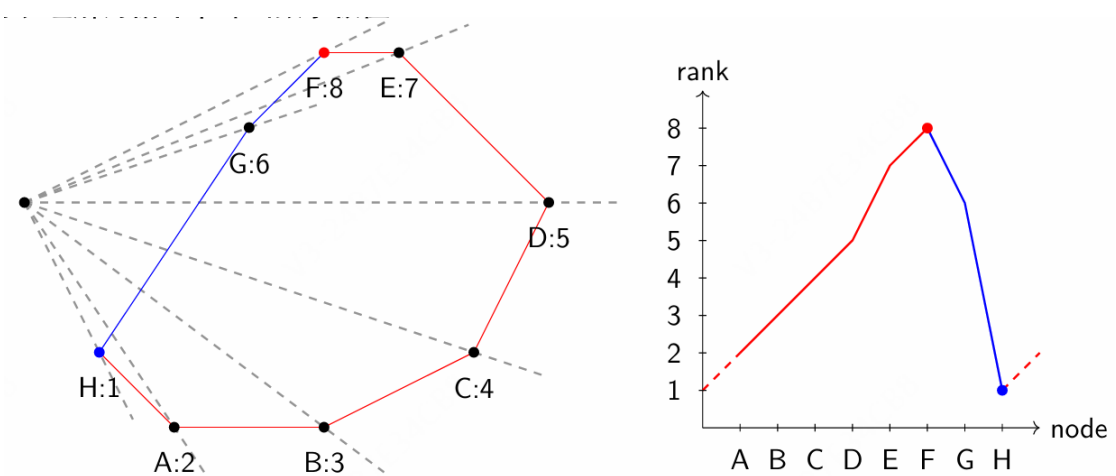
给定凸包和起点 st，希望快速找到凸包的上下切点.

将向量之间的角度映射为可比较的数值，再利用凸包的单调性进行二分.

通过点积与叉积判断相对方向，将角度转换为连续的数值表示，避免使用 tan.

先确定单调区间的边界，再利用相邻函数值的变化趋势寻找极值位置（两次二分）.

最终将二分得到的下标映射回原凸包，返回对应的切线候选顶点.



```
T getSum(pt a, pt b, int flag){ // 这个可以改，用于不同的二分场景，比如最近距离
    T x = dot(a, b);
    T y = flag ? cross(a, b) : cross(b, a);
    T z = y / (fabsl(x) + fabsl(y));
    return x >= 0 ? z : (y >= 0 ? 2 - z : -2 - z);
}
pt san(vector<pt>& con, pt st, pt q, int flag){ // flag=0: 上切线, flag=1: 下切线
    int n=con.size();pt b = q + con[1] + con[n-1] - st * 3; // 取重心处理边上的点
    auto f=[&](int i){return getSum(con[i]-st, b,flag);};
    int k=f(1)<f(n-1)?1:-1;
    auto g=[&](int i){return f((i*k%n+n)%n);};
    int R=n-1,l=1,r=R;
    if(g(R)>=g(0)){
        while(l<r){int m=(l+r)>>1; if(g(m)>=g(0)) r=m; else l=m+1;}
        R=l;}   l=0,r=R;
    while(l<r){int m=(l+r)>>1; if(g(m)<=g(m+1)) r=m; else l=m+1;}
    return con[(l * k % n + n) % n];
}
```

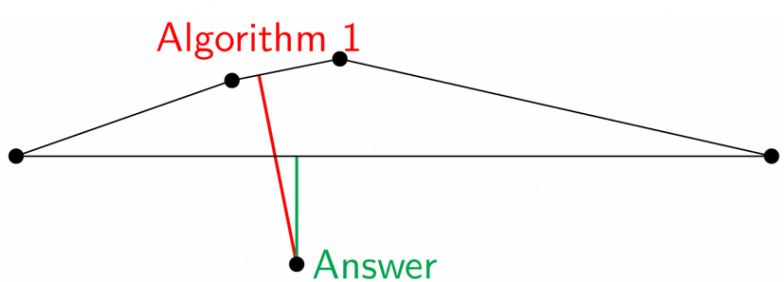
问题描述：求出目标点在凸包外时到凸包的距离

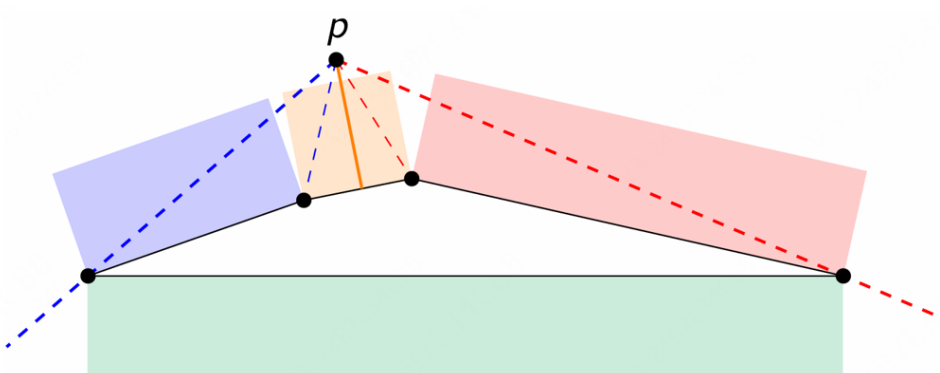
Algorithm1: 使用凸包上二分，计算凸包上离目标点最近的顶点，

然后查询这个顶点附近的边最近距离。

但这是错误的（参考左图），

正解：使用凸包切线，然后在可见链上二分最近顶点.





问题描述：给定两个凸多边形 A, B，求它们的闵可夫斯基和.

定义：$A \oplus B = \{a + b \mid a \in A,\ b \in B\}$. 就是 A 和 B 保持接触的所有位置的集合.

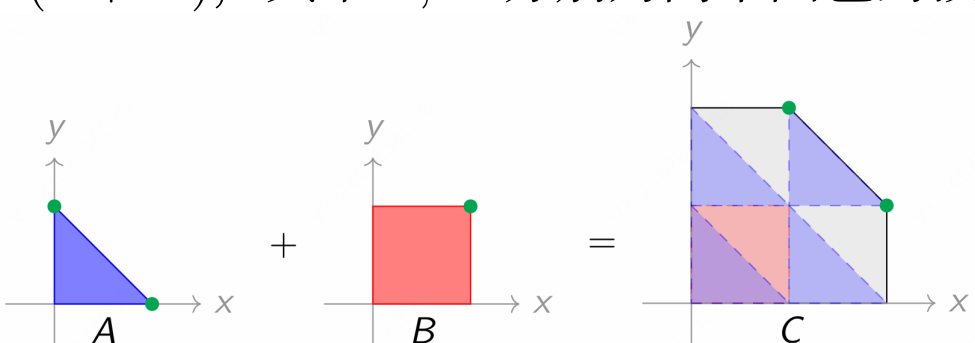
性质：两个凸多边形的闵可夫斯基和仍然是凸多边形.

分别找到两个凸包的最低点，并将其作为起点.

将两个凸包的边向量按照极角顺序归并，依次累加得到新顶点.

通过叉积比较边向量的极角，相同方向的边可以同时推进.

时间复杂度：$O(n + m)$，其中 n, m 分别为两个凸包的顶点数.



```
vector<pt> minkowski(vector<pt> a, vector<pt> b){ // 输入的必须是逆时针凸包
    auto init = [](vector<pt>& p){
        rotate(p.begin(), min_element(p.begin(), p.end(),
            [](pt a, pt b){return tie(a.y,a.x)<tie(b.y,b.x);}), p.end());
    };
    init(a); init(b);
    int n=a.size(), m=b.size(), i=0, j=0;
    vector<pt> res;
    while(i<n || j<m){
        res.push_back(a[i%n]+b[j%m]);
        T c=i==n?-1:j==m?1:cross(a[(i+1)%n]-a[i%n],b[(j+1)%m]-b[j%m]);
        if(c>=0) i++;
        if(c<=0) j++;
    }
    return res;
}
```

问题描述：给定 n 个点，求其中一对距离最大的点。输出这个距离。

暴力枚举点对需要 $O(n^2)$，能否优化？最远点对一定在凸包上！

枚举凸包的每条边 P_iP_{i+1}，维护对踵点 P_j。

固定底边，叉积越大，点到直线的距离越远。

若 P_{j+1} 的叉积更大，则将 j 向前移动。

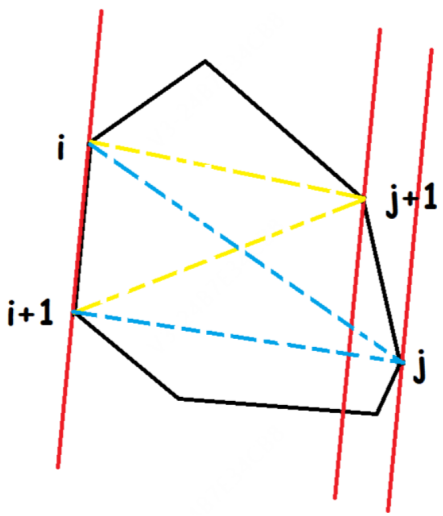
由于凸包的凸性，最远点随边旋转单调移动。

用 P_i, P_{i+1} 与 P_j 更新最大距离。

两个指针均只需遍历一圈，复杂度为 $O(n)$。

还可推广到最小面积外接矩形等问题。

这种双指针的方法我们称为旋转卡壳。这种思想将会非常常用，因此没有固定板子。



问题描述：给定 n 个点，求其中一对距离最大的点。输出这个距离。

暴力枚举点对需要 $O(n^2)$，能否优化？最远点对一定在凸包上！

其实我们还可以使用闵可夫斯基和解决这个问题。

设原凸包为 H，构造关于原点对称的凸包 $-H$。

计算闵可夫斯基和 $S = H + (-H)$。

由于 $S = \{a - b \mid a, b \in H\}$，包含了所有点对的差向量。

因此，最远点对的距离就是 S 中模长最大的向量的长度。

模长最大值一定能在凸包顶点处取得，因此只需枚举顶点。

利用边向量的极角归并，可以在线性时间内求出闵可夫斯基和，最终复杂度为 $O(n)$

问题描述：给定 n 个半平面，求它们的公共交集。

半平面交一定是凸集，若交集有界，则可以形成凸多边形。

暴力枚举直线交点复杂度较高，考虑使用双端队列优化。

首先按照直线方向的极角排序，统一规定保留直线左侧。

按照极角顺序依次加入直线，维护可能构成交集边界的直线。

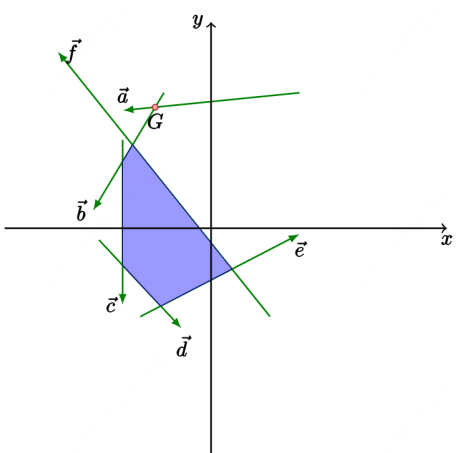
若队尾两条直线的交点不在新半平面内，则弹出队尾。

同理检查队首，不断删除不再参与交集边界的直线。

对于同向平行直线，只需要保留约束更严格的一条。

处理完毕后检查首尾，求出相邻直线的交点，得到最终凸包。

每条直线最多入队、出队一次，总复杂度为 $O(n \log n)$。

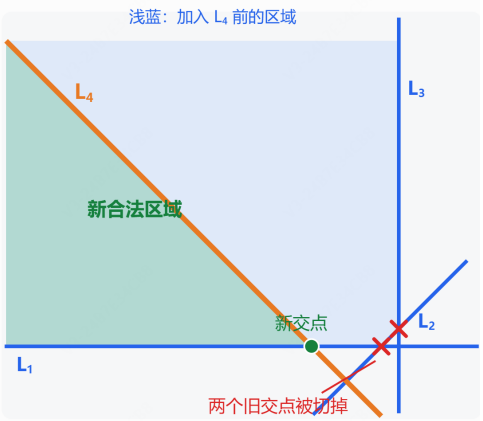


第一种情况：弹出队尾

如右图，加入 L_4 时，L_2, L_3 的交点不在左侧，弹出 L_3。

再检查 L_1, L_2 的交点，若仍不在 L_4 左侧，继续弹出队尾。

只剩一条直线时停止，最后加入 L_4。



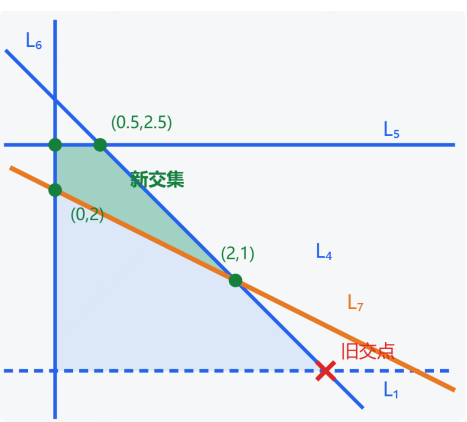
第二种情况：弹出队首

如右下图，加入新直线时，同样检查队首两直线的交点。

加入 L_7 后，L_1, L_4 的交点不在左侧，弹出 L_1。

继续检查新的队首交点，若仍不合法，就继续弹出队首。

直到交点合法或仅剩一条直线，最后加入新直线。



```
vector<pt> half_plane(vector<line> a) {
    const T eps = 1e-12L;
    for (auto &l : a) { T k = absv(l.v); l.v = l.v/k; l.c /= k; }
    sort(a.begin(), a.end(), [](line x, line y) {
        return half(x.v) != half(y.v) ? half(x.v) < half(y.v) : cross(x.v,y.v) > 0;
    });
    auto out = [&](line l, line x, line y) {
        pt p; return inter(x,y,p) && side(l,p) < -eps;
    };
    deque<line> q;
    for (line l : a) {
        while (q.size()>1 && out(l,q.back(),q[q.size()-2])) q.pop_back();
        while (q.size()>1 && out(l,q[0],q[1])) q.pop_front();
        if (!q.empty() && abs(cross(q.back().v,l.v)) < eps) {
            if (dot(q.back().v,l.v) < 0) return {};
            if (l.c <= q.back().c) continue;
            q.pop_back();
        }
```

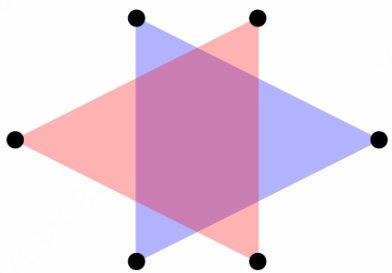
```
        q.push_back(l);
    }
    while (q.size()>2 && out(q.front(),q.back(),q[q.size()-2])) q.pop_back();
    while (q.size()>2 && out(q.back(),q[0],q[1])) q.pop_front();
    vector<pt> res;
    if (q.size()<3) return res;
    for (int i=0; i<(int)q.size(); i++) {
        pt p; if (!inter(q[i],q[(i+1)%q.size()],p)) return {};
        res.push_back(p);
    }
    return res;
}
```

注意：此代码无法区分“无界交集”和“空集”，需要的话可以补个超大矩形。

半平面交算法很少用到，实际也很容易写错（或者不优）

问题描述：给定两个凸包，判断他们是否存在公共点。

Wrong Answer: 存在顶点在另一个凸包内不是必要条件（如下图）。



正确解法 1：将两个凸包的边作为半平面，求半平面交。

这很好理解，凸包的边界就是半平面交的边界，但是有没有更优雅的方法呢?

正确解法 2：求出 A 和 −B 的闵可夫斯基和，判定零点是否在里面。

至于为什么可以这样做，自己想想吧。

欧拉公式：对于连通平面图，有 $V-E+F=2$。

其中 V 为顶点数，E 为边数，F 为面数（包括外部面）。

对于不连通平面图，若有 C 个连通分量，则 $V-E+F=C+1$。

重要推论：简单平面图中，$V\geq 3$ 时有 $E\leq 3V-6$。

简单平面图一定存在度数不超过 5 的顶点，边数为 $O(V)$。

皮克定理：对于顶点坐标均为整数的简单多边形，有 $S=i+\frac{b}{2}-1$，

其中 i 为内部整点数，b 为边界整点数。

边界整点数：每条边贡献 $\gcd(|\Delta x|,|\Delta y|)$，累加得到 b。

利用叉积求出面积 S，再通过 $i=S-\frac{b}{2}+1$ 求内部整点数。

增量法：将输入对象逐个加入，维护当前已加入对象的答案或几何结构。

假设已解决前 $i-1$ 个对象的问题，再考虑加入第 i 个对象后如何更新。

若新对象不影响当前结果，直接跳过；否则调整受影响部分，必要时重新求解。

为什么随机化？ 固定的加入顺序可能反复触发昂贵更新，导致很高的总开销。

先将输入等概率随机打乱，再做增量维护，通常可以降低昂贵更新的期望次数。

逆向分析：固定前 i 个对象的集合，把最后加入的对象看作从中均匀随机选取。

若至多 b 个对象在删除后会改变答案，则本次需要更新的概率至多为 b/i。

期望总开销由各步的“检查开销 + 更新概率 × 条件期望更新开销”累加得到。

随机顺序影响运行时间，算法仍需始终维护正确结果；具体复杂度要结合问题分析。

典型应用：最小覆盖圆、凸包构造与 Delaunay 三角剖分等。

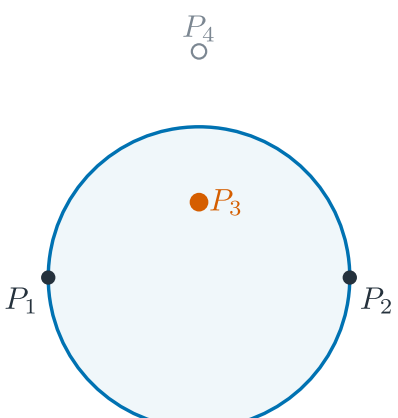
问题描述：给定 n 个平面点，求覆盖所有点的最小半径圆。

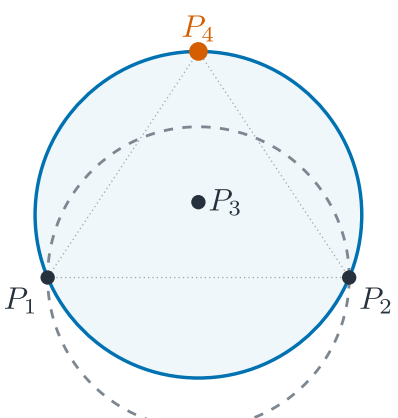
圆由至多三个点确定：单点半径为 0、两点为直径，或三点的外接圆。

随机加入 P_i：圆内或圆周上则不变；圆外则固定 P_i 在新圆圆周上，重新求解。

重算时，从半径 0 开始扫描 $j < i$；若 P_j 在圆外，先取以 P_iP_j 为直径的圆。

固定 P_i, P_j 扫描 $k < j$；若 P_k 在圆外，更新为 $\triangle P_iP_jP_k$ 的外接圆。





复杂度分析

重算概率至多为 $3/i$，

每次重算的期望开销为 $O(i)$。

故总期望 $O(n)$，最坏 $O(n^3)$。

图示：P_3 在圆内不变，P_4 在圆外重算。

实现时先用 `shuffle` 打乱；用距离平方判圆内，并为浮点比较保留误差。

返回目录

```
pair<pt,T> minCircle(vector<pt> a){
    if(a.empty()) return {{0,0},0};
    shuffle(a.begin(),a.end(),mt19937(114514));
    pt o=a[0]; T r=0;
    auto out=[&](pt p){return sq(p-o)>r;};
    for(int i=0;i<a.size();i++) if(out(a[i])){
        o=a[i]; r=0;
        for(int j=0;j<i;j++) if(out(a[j])){
            o=(a[i]+a[j])/2; r=sq(a[i]-o);
            for(int k=0;k<j;k++) if(out(a[k])){
                if(orient(a[i],a[j],a[k]))
                    o=circumCenter(a[i],a[j],a[k]);
                else o=sq(a[i]-a[k])>sq(a[j]-a[k])
                    ?(a[i]+a[k])/2:(a[j]+a[k])/2;
                r=max({sq(a[i]-o),sq(a[j]-o),sq(a[k]-o)});
            } } }
    return {o,sqrtl(r)};
}
```

04

杂题选讲

返回目录

假设海岸线是一条无限延伸的直线．它的一侧是陆地，另一侧是海洋．每一座小岛是在海面上的一个点．雷达必须安装在陆地上（包括海岸线），并且每个雷达都有相同的扫描范围 d．你的任务是建立尽量少的雷达站，使所有小岛都在扫描范围之内．

$1 \leq n \leq 10^3$.

假设海岸线是一条无限延伸的直线．它的一侧是陆地，另一侧是海洋．每一座小岛是在海面上的一个点．雷达必须安装在陆地上（包括海岸线），并且每个雷达都有相同的扫描范围 d．你的任务是建立尽量少的雷达站，使所有小岛都在扫描范围之内．

$1 \leq n \leq 10^3$.

简单题.

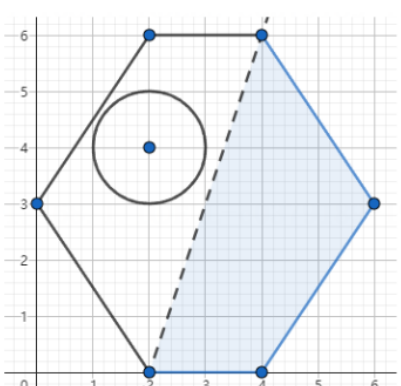
由于雷达必须安装在 x 轴上，那么对于每个小岛，覆盖它的雷达的圆心坐标是 x 轴上的一段区间．相当于有 n 个区间，选择尽量少的点，使得每个区间内都至少有一个点.

按照右端点排序后贪心选择即可.

时间复杂度 $O(n \log n)$.

包包为您准备了一个披萨！这个披萨是一个凸多边形，它的所有边缘都有馅饼皮。不过，饼皮很脆弱，只能从顶点切开，不能切到边缘中间。不幸的是，披萨上有一块相当大的圆形菠萝，你绝对要避开它。计算一次直切可以得到的最大一块无菠萝披萨，并输出其大小。保证菠萝和披萨的交点面积为 0.

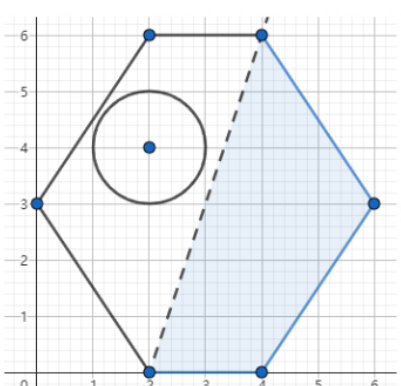
$1 \leq n \leq 10^5$.



包包为您准备了一个披萨！这个披萨是一个凸多边形，它的所有边缘都有馅饼皮。不过，饼皮很脆弱，只能从顶点切开，不能切到边缘中间。不幸的是，披萨上有一块相当大的圆形菠萝，你绝对要避开它。计算一次直切可以得到的最大一块无菠萝披萨，并输出其大小。保证菠萝和披萨的交点面积为 0.

$1 \leq n \leq 10^5$.

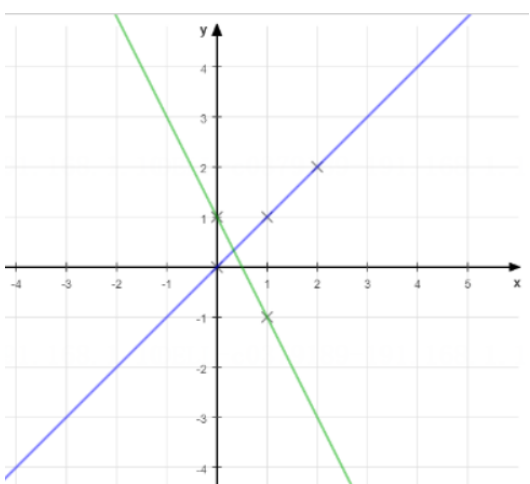
类似于旋转卡壳的思路去做就行，对于每个端点，假如当前部分没有够到菠萝就继续往前伸。注意 dist 判断不能使用开方，否则会被卡精度。



时间复杂度 $O(n)$.

返回目录

在直角坐标平面上有 n 个点。每个点都是网格点（即其坐标都是整数），且所有点都是不同的。
您可以画两条直线（不一定完全相同）。能否做到每个点至少位于其中一条直线上
$1 \leq n \leq 10^5$.



在直角坐标平面上有 n 个点。每个点都是网格点（即其坐标都是整数），且所有点都是不同的。
您可以画两条直线（不一定完全相同）。能否做到每个点至少位于其中一条直线上
$1 \le n \le 10^5$.

首先寻找不共线的三个点。可以先固定两个点后枚举找到。

这三个不共线的点里面一定有一个两个点会构成目标直线之一。于是我们只需要枚举一下这三个点中二个点的可能并擦去在这个线上的点，看看剩下的点能不能收在一条直线上就行。

时间复杂度 $O(n)$.

二维平面内有 n 个星星，每个星星都有一个亮度 b_i，问能否通过旋转一次坐标系，使得所有星星的亮度沿 y 轴从上到下依次减少或不变.

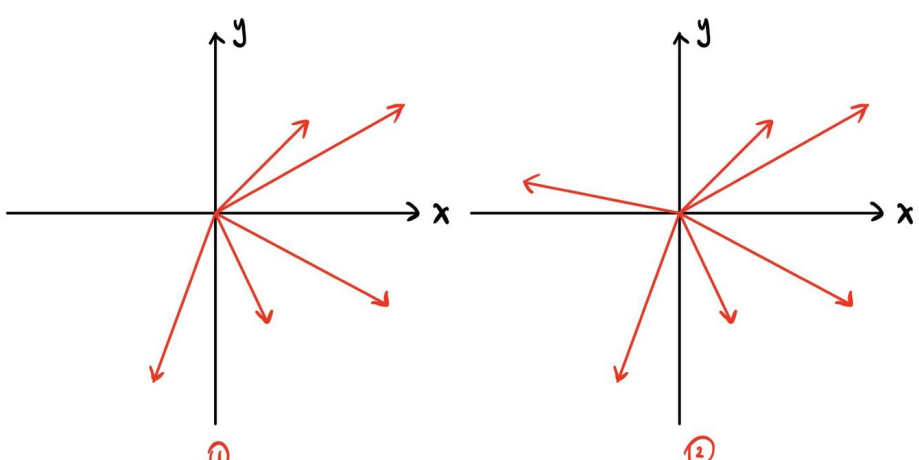
$1 \leq n \leq 1000$，精度要求比较高.

二维平面内有 n 个星星，每个星星都有一个亮度 b_i，问能否通过旋转一次坐标系，使得所有星星的亮度沿 y 轴从上到下依次减少或不变.

$1 \le n \le 1000$，精度要求比较高.

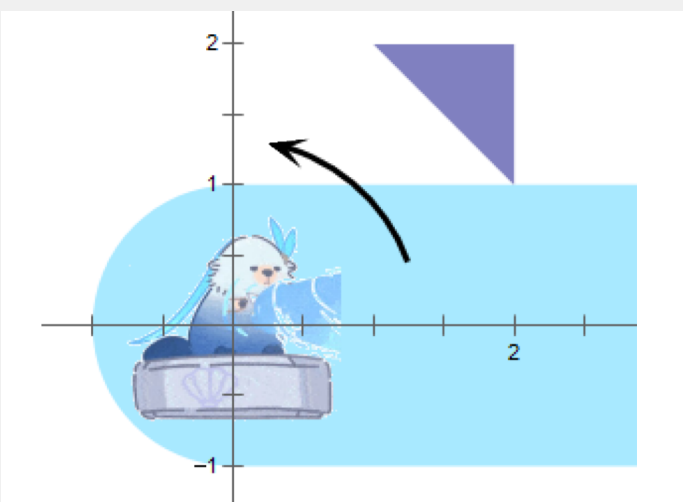
枚举每个点对. 如果 p_i 的亮度大于（没有等于）p_j，则将向量 $\vec{v} = \overrightarrow{p_i p_j}$ 加入集合.

极角排序后，看所有向量是否可以规范在一个半平面内即可.（下图 1 可以，2 不行）



水神的攻击范围是一条以原点为端点的射线，和所有与该射线距离 $\leq d$ 的点的集合（见下图）．射线的初始方向给定，会以每单位 1 弧度的速度旋转．给定一个 n 个定点的凸多边形（保证每个节点距原点 $> d$），求 $[0, t]$ 的时间内攻击范围与目标有交的时间区间长度.

$1 \leq n \leq 100$.



将凸多边形向外扩张 d 的区域，即将每个顶点扩展为一个半径为 d 的圆，并连接形成一个顶点圆滑的凸多边形.

攻击范围与原多边形相交 $\iff$ 射线与新多边形相交.

为了求解可行的极角区间，只需要求解与新多边形相交的最早与最晚位置即可．因为保证了凸多边形任意一点到原点 $> d$，因此凸多边形一定可以被划分到一个半平面内，在这个半平面内取出最靠左和最靠右的切线即可．n 比较小，遍历求切线即可.

由于 t 可能大于 2π，将 $[0, t]$ 拆分成若干个完整的 $[0, 2\pi]$ 和一个不整的 $[0, \theta]$ 即可求解答案.

时间复杂度 $O(n)$.

给定两个凸多边形 P, Q，顶点均按逆时针顺序给出，顶点数分别为 n, m。
允许任意平移 Q，但不能旋转或翻转。将 P 与平移后的 Q 的交集面积记为伤害。
只考虑伤害严格大于 0 的平移，在这些平移向量构成的区域内均匀随机选取。
求碰撞伤害的期望值。

记 $f(t) = S(P \cap (Q + t))$，合法平移区域为 $D = \{t \in \mathbb{R}^2 \mid f(t) > 0\}$。

$3 \le n, m \le 1000$，各测试的 $\sum n, \sum m \le 1000$，坐标绝对值不超过 10^4。

输出期望值，绝对误差或相对误差不超过 10^{-6}。

$$\mathbb{E}[f(t)] = \frac{1}{S(D)} \iint_D f(t)\,\mathrm{d}t$$

分母：合法平移区域为 $D = \mathrm{int}(P + (-Q))$，面积即闵可夫斯基和的面积。

因为 $P \cap (Q + t) \neq \varnothing \iff t = p - q$，其中 $p \in P, q \in Q$；仅接触落在边界上。

分子：固定 $x \in P$，使 $x \in Q + t$ 的平移集合为 $x - Q$，面积恰为 $S(Q)$。

将“对所有平移累加交集面积”改为“对 P 中每个点累加被覆盖的平移面积”。

交换积分次序；区域 D 外交集面积为 0，因此可以对整个平面积分：

$$\iint_{\mathbb{R}^2} f(t)\,\mathrm{d}t = \iint_P \left(\iint_{\mathbb{R}^2} \mathbf{1}_{Q+t}(x)\,\mathrm{d}t \right) \mathrm{d}x = S(P)S(Q)$$

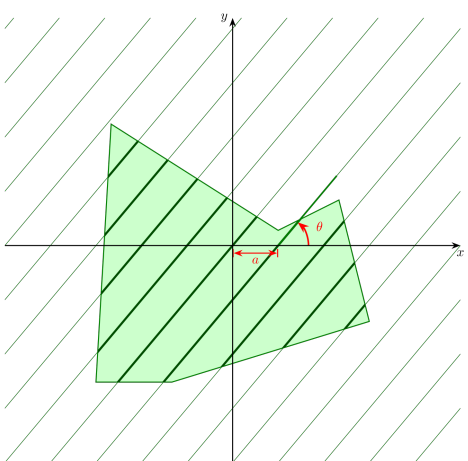
$$\boxed{\mathbb{E}[f(t)] = \frac{S(P)S(Q)}{S(P + (-Q))}}$$

若使用二倍面积 A_P, A_Q, A_R，其中 $R = P + (-Q)$，则答案为 $A_P A_Q / (2A_R)$。

极角归并求闵可夫斯基和，复杂度 $O(n + m)$.

给定按顺时针顺序描述的简单多边形，求它内部所有竹子线段的总长度。
竹子为无穷多条平行直线：第 k 条经过 $(ka, 0)$，与 x 轴夹角为 θ，$k \in \mathbb{Z}$。
多边形可以是凹的；保证任意一条边都不与竹子平行。

$n \le 10^5$，$|x_i|, |y_i| \le 10^4$，$\theta \in [1°, 179°)$，$a \in [0.1, 100]$。
输出总长度，绝对误差或相对误差不超过 10^{-6}。



注意：a 是相邻竹子在 x 轴上交点的间距。

相邻直线的垂直距离为 $a \sin \theta$。

旋转坐标：令 $u = x\sin\theta - y\cos\theta$，$v = x\cos\theta + y\sin\theta$，旋转保持长度。

竹子变为竖直线 $u = kh$，其中 $h = a\sin\theta$，问题变为累加竖直截线的长度。

有向三角剖分：固定原点 O，沿每条有向边构造三角形 $\triangle OP_iP_{i+1}$。

三角形按叉积符号加减，外部区域和重叠部分自动抵消，凹多边形也可以这样处理。

$$\mathrm{Ans} = \left|\sum_{i=1}^{n} \sigma_i F_i\right|, \qquad \sigma_i = \mathrm{sgn}(P_i \times P_{i+1})$$

其中 F_i 为第 i 个三角形中的竹子总长，$P_{n+1} = P_1$；退化三角形贡献为 0。

将单个三角形的顶点按 u 排序为 L, M, R，横坐标分别为 $l \le m \le r$。

中间截线长度 $H = |(M - L) \times (R - L)|/(r - l)$，截线长度分两段线性变化。

从 l 到 m 线性增加，从 m 到 r 线性减少，因此可以分别按等差数列求和。

必须带符号累加后统一取绝对值，不能把各三角形的贡献全部当成正数相加。

设 $\ell(u)$ 为单个三角形在横坐标 u 处的截线长度，则

$$\ell(u) = \begin{cases} H\dfrac{u-l}{m-l}, & l \le u < m, \\ H\dfrac{r-u}{r-m}, & m \le u < r. \end{cases}$$

空区间直接跳过，避免除以 0；其余每段都可以写为 $\ell(u) = \alpha u + \beta$。

对于区间 $[p, q)$，竹子下标范围为 $s = \lceil p/h \rceil$ 至 $t = \lceil q/h \rceil - 1$。

$$\sum_{k=s}^{t} \ell(kh) = \alpha h \frac{(s+t)(t-s+1)}{2} + \beta(t-s+1)$$

若 $s > t$，该段没有竹子，贡献为 0；否则在 $O(1)$ 时间内完成求和。

统一采用左闭右开区间，避免剖分边恰与竹子重合时重复计算。

总时间 $O(n)$。

返回目录

平面上给定 n 个互不相同的整点，它们位于 k 条互不重合的平行直线上。
每条直线上至少有两个点，但这些直线的位置与方向均未知。
请将所有点分成 k 组，使每组点共线，且各组所在的直线互相平行。
输出每组的点数及点的编号，每个点恰好出现一次；保证有解，多解输出任意一种。

$2 \le n \le 10^4$，$1 \le k \le \min(50, \lfloor n/2 \rfloor)$，$1 \le x_i, y_i \le 10^9$。

随机猜方向，再检查。

1. 随机选择两个不同的点 A, B，取方向向量 $d = B - A = (d_x, d_y)$。

2. 对每个点计算 $c_i = d \times P_i = d_x y_i - d_y x_i$，按 c_i 排序、分组。

c_i 相同的点就在同一条平行于 d 的直线上，竖直方向也同样适用。

3. 若恰有 k 组，且每组至少两个点，就输出；否则重新随机选择 A, B。

随机两点落在同一条目标直线上的概率为 $\Omega(1/k)$，平均尝试 $O(k)$ 次即可。

每次排序、检查需要 $O(n \log n)$，总期望时间复杂度为 $O(nk \log n)$。

保安沿 n 个点构成的闭合折线，按给定顺序巡逻，博物馆内有 m 件展品。
保安在线段内部时，视野朝向该段的前进方向，张角为 $2a$，视野半径可视为无限大。
求一圈路线中，能够同时看到所有展品的部分的总长度。

路线不要求为简单多边形；转弯处的端点不计入长度。

$3 \le n \le 2 \times 10^5$，$1 \le m \le 2 \times 10^5$，$0^\circ < a < 90^\circ$，坐标绝对值不超过 10^9。
输出总长度，绝对误差或相对误差不超过 10^{-6}。

1. 先求展品的凸包。枚举巡逻段 $A \to B$，两条视野边界方向分别为 AB 旋转 $\pm a$。

2. 对每条边界，在凸包上二分找内法向投影最小的顶点，即最限制视野的点。

这是给定方向的支撑点查询，沿用 3.1 中分单调链、二分极值的思路即可。

3. 过这两个顶点，分别作对应视野边界的平行线，与 AB 所在直线求交。

设两个交点距 A 的有向距离为 t_1, t_2，正方向为 $A \to B$，且 $L = |AB|$。

$$\text{本段贡献} = \max(0, \min(L, t_1, t_2))$$

沿 $A \to B$ 前进，视野只会缩小，所以合法部分是从 A 出发的一段前缀。

将所有巡逻段的贡献相加即可；凸包只有一个点或一条线段时直接处理。

预处理 $O(m \log m)$，每段两次 $O(\log m)$ 查询，总复杂度 $O(m \log m + n \log m)$。

05

后记

返回目录

我们介绍了一些几何工具，以及对应实现。

一个问题可能有多种解决方法。

选择正确的工具解决问题比会做题本身更重要。

判断题目难度比会做题本身更重要。

「菜就多练」

如果有志于在比赛里解决几何题，多训练有几何的场。

如果只补过题而没有在训练里通过几何题，比赛现编是不现实的。

「输不起就别玩」

开几何通常是比赛的胜负手。

这个决策必须有着绝对信任和责任。

不开几何比乱开几何好。

实现难度、精度、运行效率、泛用性是无法兼得的。
最短的、最快的代码不一定是最好的代码。
自己熟悉，并且能做出有效修改的才是最好的代码。

计算几何题通常也会有大量边界情况与分类讨论。
因此，一个通常正确的策略也许是推迟实现几何题。
反其道而行之：几何题容易抢一血一球?

在超越自己掌握的模板库时，一个通常正确的决策是放弃几何题。
在水平 $< L$ 或者 $> R$ 的时候，一道题目不会成为队伍的负担。
在水平 $< R$ 但以为自己 $> R$ 时，容易引发事故。
即使掌握，也要当作带有随机性、有精度问题、有分类讨论的大模拟题来谨慎处理。

即使没用到随机算法，随机性可能来自：数据太弱、太强、有错、评测机环境 ……

准确感受自己的水平!

2024 省赛/邀请赛

题号

2024 湖北省赛 B

2024 江苏省赛 B

2024 CCPC 河南邀请赛 D

2024 四川省赛 F

2024 CCPC 长春邀请赛 D

2024 江西省赛 I

2024 CCPC 东北邀请赛 M

2024 CCPC 济南邀请赛 M

2024 区域赛

题号

2024 ICPC 南京 M

2024 ICPC 昆明 H

2024 ICPC 沈阳 G

2025 省赛/邀请赛（1/2）

题号

2025 湖南省赛 B

2025 湖南省赛 J

2025 CCPC 南昌 I

2025 CCPC 郑州 L

2025 广西省赛 K

2025 山东省赛 M

2025 吉林省赛 B

2025 新疆省赛 J

2025 省赛/邀请赛（2/2）

题号

2025 河南省赛 I

2025 河南省赛 K

2025 ICPC 武汉邀请赛 M

2025 浙江省赛 E

2025 浙江省赛 M

2025 区域赛（1/2）

题号
2025 ICPC 网络赛 Ⅰ J
2025 ICPC 网络赛 Ⅱ A
2025 ICPC 武汉 G
2025 ICPC 成都 I
2025 ICPC 南京 B
2025 ICPC 南京 M
2025 ICPC 沈阳 G
2025 ECF 杭州 A

2025 区域赛（2/2）

题号

2025 CCPC 哈尔滨 B

2025 CCPC 哈尔滨热身赛 D

2025 ICPC 沈阳热身赛 D

2026 省赛/邀请赛

题号

2026 ICPC 沈阳邀请赛 J

2026 重庆市赛 K

2026 山东省赛 M

2026 CCPC 南昌邀请赛 J

2026 江苏省赛 D

2026 ICPC 深圳邀请赛 M

2024 省赛/邀请赛

题号	难度	标签
2024 湖北省赛 B	118/168，Cu-	枚举
2024 江苏省赛 B	25/190，Au-	数学
2024 CCPC 河南邀请赛 D	37/442，Au	距离转化，思维
2024 四川省赛 F	42/284，Ag+	分类讨论
2024 CCPC 长春邀请赛 D	8/180，Au+	随机化
2024 江西省赛 I	0/184，防 AK 题	分类讨论
2024 CCPC 东北邀请赛 M	30/260，Au	分类讨论，枚举
2024 CCPC 济南邀请赛 M	1/274，捧杯	dp

2024 区域赛

题号	难度	标签
2024 ICPC 南京 M	9/336，Au+	极角求并，距离转化
2024 ICPC 昆明 H	342/450，签到 +	极角排序
2024 ICPC 沈阳 G	43/300，Au-	思维，交互

2025 省赛/邀请赛（1/2）

题号	难度	标签
2025 湖南省赛 B	？，Ag	仿射变换
2025 湖南省赛 J	？，Au+	分类讨论，枚举
2025 CCPC 南昌 I	0/259，防 AK	思维
2025 CCPC 郑州 L	2/433，捧杯题	凸包，双指针
2025 广西省赛 K	23/218，Au-	分类讨论
2025 山东省赛 M	0/231，防 AK	思维，构造，贪心
2025 吉林省赛 B	5/175，捧杯题	双指针，数学
2025 新疆省赛 J	2/108，捧杯题	双指针，数学

2025 省赛/邀请赛（2/2）

题号	难度	标签
2025 河南省赛 I	57/163，Ag	凸包
2025 河南省赛 K	47/163，Ag	枚举
2025 ICPC 武汉邀请赛 M	26/280，Au	高维几何，分类讨论
2025 浙江省赛 E	0/400，防 AK	积分，双指针，不可做
2025 浙江省赛 M	24/400，Au+	三角剖分，思维

2025 区域赛（1/2）

题号	难度	标签
2025 ICPC 网络赛 Ⅰ J	127/2161，Au-	距离转化，容斥
2025 ICPC 网络赛 Ⅱ A	131/1756，Ag+	高维几何，思维，凸包
2025 ICPC 武汉 G	10/446，Au+	二分，思维，高维几何
2025 ICPC 成都 I	17/320，Au	双指针，差分，凸包切线
2025 ICPC 南京 B	40/335，Au-	极角排序，强思维，半平面
2025 ICPC 南京 M	27/335，Au	NTT 卷积，叉积运用
2025 ICPC 沈阳 G	33/400，Au	闵可夫斯基和，期望
2025 ECF 杭州 A	66/280，Ag	分数处理，分类讨论

2025 区域赛（2/2）

题号	难度	标签
2025 CCPC 哈尔滨 B	74/280，Ag	模拟，分类讨论
2025 CCPC 哈尔滨热身赛 D	30/250，Ag+	三角剖分
2025 ICPC 沈阳热身赛 D	21/383，Au	Green 公式，分类讨论

2026 省赛/邀请赛

题号	难度	标签
2026 ICPC 沈阳邀请赛 J	51/281，Ag+	单调栈
2026 重庆市赛 K	73/218，Ag-	单调栈
2026 山东省赛 M	2/237，捧杯	凸包切线
2026 CCPC 南昌邀请赛 J	9/307，Au+	极角排序，二分，双指针
2026 江苏省赛 D	3/178，捧杯	凸包，极角求并
2026 ICPC 深圳邀请赛 M	17/276，Au+	分类讨论

References

- https://victorlecomte.com/cp-geo.pdf
- https://fstqwq.pw/Computationnal_Geometry_2024.pdf
- 本 PPT 原作者：memset0
- 协作者：ChatGPT-6.1 Sol (OpenAI)